School of Computer Science
FACULTY OF ENGINEERING AND PHYSICAL SCIENCES

UNIVERSITY OF LEEDS

Final Report

Computational and Graphical Simulation of Photovoltaic Panels

Drew Evans

Submitted in accordance with the requirements for the degree of MEng, BSc Computer Science

2025/2026

COMP3931 Individual Project

The candidate confirms that the following have been submitted.

Items	Format	Recipient(s) and Date
Final Report	PDF file	Uploaded to Minerva (29/04/26)
Link to online code repository	URL	Sent to supervisor and assessor (29/04/26)

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of Student) Drew Evans

© 2025/2026 The University of Leeds and Drew Evans

Summary

There is a difficult problem surrounding predicting the output (power in watts) of photovoltaic modules (i.e. solar panels) under partially shaded conditions. This project aims to demonstrate that combined computational and graphical methods can be used to accurately and efficiently predict this output. The software is first implemented in `Python`, to validate the computational methods, followed by the implementation of a 3D graphical model in `Unity` to simulate dynamic shade and sunlight. A series of mathematical equations are established to estimate physical parameters of cells within a module. Subsequently, trust region solvers and artificial neural networks are utilised to estimate power output. This is linked to the graphical model. Furthermore, the performance and accuracy of these different methods are validated and tested, in order to select the best individual methods, and ensure the reliability of the overall solution.

Acknowledgements

First, I would like to thank my supervisor Dr Timon Gutleb and assessor, Dr Massimiliano Fasi, for their advice and input on this project. I would like to also thank my friends and family who supported me throughout its creation.

Contents

1 Introduction and Background Research **1**

1.1 Introduction . . . 1

1.2 Background Research . . . 1

1.2.1 I-V Curves, Bypass Diodes & the Mismatch Problem . . . 1

1.2.2 The single diode model . . . 2

1.2.3 Single Diode Parameter Calculation . . . 3

1.2.4 Heuristic Method for Parameter Extraction . . . 5

1.2.5 Artificial Neural Network (ANN) for Parameter Estimation . . . 6

1.3 Panel Mismatch Modelling . . . 7

1.3.1 Modelling as a Whole Panel Vs. as a Series of Cells . . . 7

1.3.2 Implicit Trust Region Modelling . . . 7

1.3.3 Explicit I-V Calculations & Avalanche Breakdown . . . 10

1.4 Graphical Modelling and Irradiance . . . 11

1.5 Existing Models & Limitations . . . 11

2 Methodology **13**

2.1 Design of Proposed Solution . . . 13

2.1.1 The Research Gap & Project Overview . . . 13

2.1.2 Project Management . . . 13

2.1.3 Proposed Development Pipeline . . . 13

2.1.4 System Design . . . 14

2.2 Validation Plan & Version Control . . . 16

2.2.1 Version Control . . . 16

2.2.2 Validation Plan . . . 16

2.2.3 Unit Testing & Continuous Integration (CI) Pipeline . . . 16

3 Implementation and Validation **17**

3.1 Pipeline 1 - Predicting Cell Parameters . . . 17

3.1.1 Reference Parameter Extraction . . . 17

3.1.2 Operational Parameter Calculations . . . 18

3.1.3 Heuristic Method . . . 18

3.1.4 The Single Cell Class . . . 19

3.1.5 Implementing the ANN for Cell Parameter Prediction . . . 19

3.2 Pipeline 1 Validation & Performance . . . 20

3.2.1 Parameter Extraction Method . . . 20

3.2.2 Single Cell Validation . . . 20

3.2.3 Prediction Profiling . . . 21

3.3 Pipeline 2 - Panel Mismatch Solvers . . . 21

3.3.1 The Panel Class . . . 21

3.3.2 Implicit Modelling . . . 22
3.3.3 Explicit Modelling . . . 23
3.4 Pipeline 2 - Validation . . . 24
3.4.1 Implicit & Explicit Curve Validation . . . 24
3.4.2 MPPT Validation . . . 24
3.5 Pipeline 3 - Graphical Irradiance Gathering . . . 25
3.5.1 Reverse Ray-Casting . . . 25
3.5.2 The UDP Server . . . 26
3.5.3 The Combined Process . . . 26
3.6 Pipeline 3 - Validation . . . 26

4 Results, Evaluation and Discussion 27
4.1 Results . . . 27
4.1.1 Panel Parameter Extraction . . . 27
4.1.2 Single Cell & ANN Prediction Results . . . 27
4.1.3 Whole Module Results . . . 28
4.1.4 Combined Process Results . . . 29
4.2 Evaluation . . . 29
4.3 Future Work . . . 30

References 31

Appendices 34

A Self-appraisal 34
A.1 Critical self-evaluation . . . 34
A.2 Personal reflection and lessons learned . . . 34
A.3 Legal, social, ethical and professional issues . . . 35
A.3.1 Legal issues . . . 35
A.3.2 Social issues . . . 35
A.3.3 Ethical issues . . . 36
A.3.4 Professional issues . . . 36

B External Material 38

C Supplementary Figures 39
C.1 Tables . . . 39
C.1.1 Values and Bounds for Heuristic Method . . . 39
C.1.2 Tracker Hyper Parameter Optimisation . . . 39

D Supplementary Graphical Results 42

Nomenclature

Standard Symbols

a	Modified diode ideality ($a = \frac{nN_s kT}{q}$)
D	Converter duty cycle
E_g	Band gap energy of the semiconductor material
G_{eff}	Effective irradiance on a panel
I	Current (A)
I_{mp}	Current at the maximum power point (P_{max}) of a panel
I_{ph}	Photocurrent (current generated by photovoltaic cells)
I_s	Reverse saturation current
I_{sc}	Short circuit current (panel current at $V = 0$)
k	Boltzmann's constant (1.3806×10^{-23} J/K)
n	Diode ideality factor
N_s	Number of cells in series
P	Power output of a panel
P_{max}	Maximum power output of a panel
q	Elementary electrical charge (1.602×10^{-19} C)
R_s	Series resistance (Ω)
R_{sh}	Shunt resistance (Ω)
S	Irradiance (W/m^2)
T	Temperature (°C or K)
V	Voltage (V)
V_{br}	Breakdown voltage
V_{mp}	Voltage at the maximum power point (P_{max}) of a panel
V_{oc}	Open circuit voltage (panel voltage at $I = 0$)

Greek Symbols

α	Temperature coefficient of I_{sc} (%/°C)
β	Temperature coefficient of V_{oc} (%/°C)
γ	Temperature coefficient of P_{max} (%/°C)
η	Converter efficiency (%)

Acronyms

STC	Standard Test Conditions of panels (1000 W/m^2 and 25°C)
WTC	Worst Test Conditions ($100W/m^2$ and 65°C)

Chapter 1

Introduction and Background Research

1.1 Introduction

Solar Panels Overview

Photovoltaic (PV) cells are devices constructed from semiconductor materials, such as silicon, that convert solar energy into electricity by using the photovoltaic effect to generate a current [1, 2]. As individual cells only produce a maximum voltage of ~ 0.7V, multiple are connected in series to form a substring. In a series configuration, the current must remain constant across all cells, and the substring's total voltage is the sum of every individual cell's voltage. A complete solar panel (or module) is constructed by connecting several substrings together in parallel [3]. The amount of solar energy striking a panel is measured as irradiance in watts per meter squared. Recently there has been significant research into how shading (which reduces this irradiance) impacts these panels. However the majority of this modelling is done at panel level, assuming it experiences uniform shade. This is unrealistic, as shading will often only impact a few individual cells on a panel. When individual cells in a substring become shaded, they become reverse-biased, causing their voltage to become negative. This subsequently reduces the overall power output of the panel, as power is a product of current and voltage [2, 4].

Project Objectives & Deliverables

The overall aim of this project is to develop a simple and efficient way to model the power output of a solar panel, based on its position and the light source. The first objective is to investigate methods to simulate the current-voltage (I-V) curve of individual solar cells, based on irradiance and temperature. The second will be to expand on the cell parameters to generate an I-V curve for the whole module, where different cells have different amounts of shading. Based on a panel level I-V curve, a power-voltage (P-V) curve can be found by multiplying each I-V point. The third objective involves developing and testing different methods to track the maximum point on this P-V curve (P_{max}). The fourth objective is to investigate how to graphically model how shade falls on a panel, to gather per cell irradiance data. From this data realistic I-V and P-V curves can be generated. Using these curves, optimal module performance can be found, given cells have varying degrees of shading.

1.2 Background Research

1.2.1 I-V Curves, Bypass Diodes & the Mismatch Problem

An I-V curve of a solar cell or panel demonstrates the inversely proportional relationship between current and voltage. As voltage increases, current decreases and vice versa.

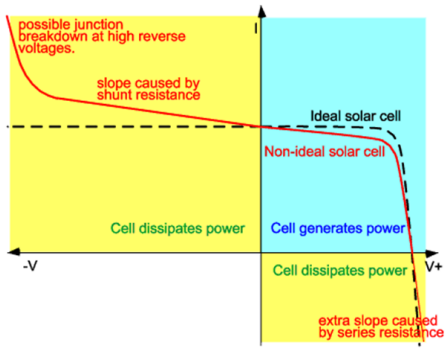


Figure 1.1: A figure showing a standard cell I-V curve with mismatch [5].

Series Mismatch

Series mismatch occurs when a cell is shaded more than the others in the same substring, which is known as short circuit mismatch. More shading means less irradiance, which means less induced current, therefore its I_{sc} (the cells maximum current) would be lower [4]. If this cell is held at a higher current it can become reverse biased and produce a negative voltage due to the nature of the I-V curve as shown in Figure 1.1 [5].

Bypass Diodes

Consider the case of a 5 cell substring, with 4 unshaded and 1 shaded. If the operating current of the string rises above the lower I_{sc} of the shaded cell, then the shaded cell will become reverse biased, causing large amounts of power to dissipate in the cell. This can cause overheating and damage the cell [6]. To avoid damage, and keep a higher panel output, bypass diodes are used. These are connected in opposite polarity to the substring and are normally reverse biased, but if a cell becomes reverse biased, the diode will begin to conduct. This allows current to flow around the string, limiting cells to a minimum voltage of -0.6V.

1.2.2 The single diode model

The Single Diode Circuit

The single diode model can be used to model the I-V curve of a solar cell or panel. The circuit structure for the single diode model is composed of a current source, a diode (an electrical component only allowing current to flow one way), a shunt and series resistance [7]. The current source is the photocurrent (I_{ph}) generated from the photovoltaic effect, the shunt resistance (R_{sh}) models the leakage current and the series resistance (R_s) represents the semiconductor resistance.

The Single Diode Equation & the 5 Parameter Model

To model the I-V curve using this circuit, Equation 1.1 shows the single diode equation, which uses 5 parameters to extract I from V or vice versa. The parameters are photocurrent (I_{ph}), reverse saturation current (I_s) diode ideality (n) which shows how closely the curve will follow the ideal curve from Figure 1.1 and R_s and R_{sh}. The rest of the terms are cell count (N_s), Boltzmann's constant (k), electrical charge (q) and temperature in kelvin (T), which make up

the thermal voltage, which affects the diode ideality. As temperature increases, the diode becomes less ideal, and the I-V curve moves away from the ideal curve [8]. When using the equation for a single cell $N_s = 1$.

$$I = I_{ph} - I_s \left[\exp\left(\frac{q(V + IR_s)}{nN_s kT} \right) - 1 \right] - \frac{V + IR_s}{R_{sh}} \tag{1.1}$$

This equation can be used to implicitly calculate an I-V curve using Newton's method. Newton's method works by taking an initial guess of the required parameter, before iterating the guess by the derivative of the function results, until it converges onto a stable solution [9]. This means that with the known 5 parameters, a series of currents can be used to find their corresponding voltages (or vice versa), via iteration.

Industry & Academic Usage of the Single Diode Model

The single diode equation used here is both the industry and academic standard for solar panels, as used by the photovoltaic System Advisor Model (SAM) from the National Laboratory of the Rockies, a leader in the field of researching and developing renewable energy sources [10]. It is used due to its high precision with lower complexity. Another potential model is the 2 diode model, which models another diode in parallel to the cell. It is generally agreed to be a more accurate way to model solar cells, and works well in all 3 sections that an I-V curve can fall into [11]. However, due to the two diode equations used to calculate the current from voltage, the model requires 7 total parameters making it more computationally expensive. Although it tends to be more accurate, the single diode model was chosen to compromise between accuracy and execution speed.

1.2.3 Single Diode Parameter Calculation

Reference Parameters

Using the single diode model to generate panel I-V curves requires the values of the 5 parameters. However, typical solar panel datasheets don't include these values, instead recording significant points on the I-V curve at Standard Test Conditions (STC - $1000W/m^2$ and 25°C) [12]. Therefore the 5 parameters need to be extracted from:

- P_{max} – optimal panel power (W)
- V_{mp} and I_{mp} – the I-V values at P_{max} (V and A)
- V_{oc} and I_{sc} – the V value when $I = 0$ and I value when $V = 0$ (V and A)

Extraction from a Simulated Curve

Tayyan proposes a method to use these known point to simulate an I-V curve, from which the 5 parameters can be approximated at STC [13]. Equation 1.2 is used to approximate a current value (I_{TA}), using coefficients calculated with Equation 1.3.

$$I_{\mathrm{TA}} = I_{\mathrm{sc}} - C_1 \exp\left(-\frac{V_{\mathrm{oc}}}{C_2} \right) \left[\exp\left(\frac{V}{C_2} \right) - 1 \right] \tag{1.2}$$

$$C_1 \cong I_{\text{sc}} \qquad C_2 \cong \frac{V_{\text{mp}} - V_{\text{oc}}}{\ln\left(1 - \frac{I_{\text{mp}}}{I_{\text{sc}}}\right)} \tag{1.3}$$

To extract the 5 parameters from this simulated curve requires an implicit solver. Extraction can be done by least square optimisation. This approach would use a rearranged single diode equation Equation 1.4, using the estimated I_{TA} values for I. Least square optimisation works by minimising the difference between simulated target variables and the actual data [14], via iterative updates. In this case, adjusting the values of the unknown 5 parameters until the equation is balanced.

$$I - I_{ph} - I_s\left[\exp\left(\frac{q(V + IR_s)}{nN_s kT}\right) - 1\right] - \frac{V + IR_s}{R_{sh}} = 0 \tag{1.4}$$

Extraction from Points & Max Power Point Derivative

Another method to extract the reference conditions only fits the 5 parameters to the datasheet I-V pairs and the derivative of the I-V curve at the P_{max} point [15]. This method involves non-linear least square optimisation, due to the 4 separate equations which require solving. The first 3 equations are Equation 1.4 evaluated at the I-V pairs: (I_{mp}, V_{mp}), (0, V_{oc}) and (I_{sc}, 0). De Soto et al. proposes a fourth equation, using the derivative of the I-V curve at the P_{max} point, as shown by Equation 1.5 [15]. At P_{max}, the derivative of the curve should be 0.

$$\left.\frac{\mathrm{d}(IV)}{\mathrm{d}V}\right|_{\text{mp}} = I_{\text{mp}} + V_{\text{mp}}\left.\frac{\mathrm{d}I}{\mathrm{d}V}\right|_{\text{mp}} = 0 \quad \text{and} \quad \left.\frac{\mathrm{d}I}{\mathrm{d}V}\right|_{\text{mp}} = \frac{-\frac{I_s}{a}e^{\frac{V_{\text{mp}}+I_{\text{mp}}R_s}{a}} - \frac{1}{R_{\text{sh}}}}{1 + \frac{I_s R_s}{a}e^{\frac{V_{\text{mp}}+I_{\text{mp}}R_s}{a}} + \frac{R_s}{R_{\text{sh}}}} \tag{1.5}$$

Operational Parameters at Actual Conditions

The previous methods extract the panels 5 parameters at STC, which are their reference values. For this project, the parameters at actual temperatures and irradiances are needed. These can be calculated given their reference values. As the panel moves away from the STC, it becomes less efficient, which is reflected in the equations of I_{ph}, R_{sh} and I_s. Equation 1.6 shows I_s increasing exponentially with temperature [15]. This is due to band gap energy (Eg – the energy required to free electrons and generate current [16]) decreasing with temperature as shown by Equation 1.6 [15].

$$\frac{I_{\text{s}}}{I_{\text{s,ref}}} = \left(\frac{T_{\text{c}}}{T_{\text{c,ref}}}\right)^3 \exp\left[\frac{1}{k}\left(\left.\frac{E_{\text{g}}}{T}\right|_{T_{\text{ref}}} - \left.\frac{E_{\text{g}}}{T}\right|_{T_{\text{c}}}\right)\right] \quad \text{and} \quad \frac{E_{\text{g}}}{E_{\text{g,ref}}} = 1 - 2.677\times10^{-4}(T - T_{\text{ref}}) \tag{1.6}$$

I_{ph} is a linear function of incident solar radiation (S), as shown by Equation 1.7, which measures how much irradiance is in contact with a panel. I_{ph} is also affected by α_{sc}, which states how much the short circuit value changes per degree of temperature change [12, 15]. The final variable M is the air mass modifier, which describes the length the radiation must travel through the air from the light source to the panel.

$$I_{\text{ph}} = \frac{S}{S_{\text{ref}}}\frac{M}{M_{\text{ref}}}\left[I_{\text{ph,ref}} + \alpha_{\text{sc}}(T_{\text{c}} - T_{\text{c,ref}})\right] \tag{1.7}$$

De Soto et al. ran experiments for R_{sh} and R_s by generating I-V curves, with varying values for the two. They found changing R_s value shifted the V_{mp} by 1% when altered by a factor of 2, while R_{sh} shifted it by 8%. Therefore, it is suggested to keep Rs as a constant, and model R_{sh} as inversely proportional to irradiance [15].

1.2.4 Heuristic Method for Parameter Extraction

Temperature Coefficient Validation

Another proposed method involves the same least square optimisation method as discussed in Section 1.2.3. The difference is that this method involves a 6th parameter, known as the adjust variable, and 3 additional datasheet values, which are the temperature coefficients of V_{oc}, I_{sc} and P_{max} (β_{oc}, α_{sc} and γ respectively) [12]. These values are used to show the change in the I-V curve points given a temperature change (%/°C). In short, if the temperature changes by 5 degrees, then the percentage change of the current and voltage of the 3 known points should match the change specified by the datasheet [16].
To ensure this match, an additional 2 equations are used. Equation 1.8 ensures that the 5 parameters are consistent given a temperature change, by finding the operational parameters at a known change (e.g. +5°C) and using an adjusted single diode equation. With this change, the expected values of I_{sc} and V_{oc} can be found from their temperature coefficients. If the parameters reference values are correct, then the equation should balance [16].

$$\left(I_{\mathrm{ph}} + \alpha_{\mathrm{sc}} \cdot \left(1 - \frac{\mathrm{Adjust}}{100}\right) \cdot \Delta T\right) - I_{\mathrm{s},T'} \left[\exp\left(\frac{V_{\mathrm{oc},T'}}{a_{T'}}\right) - 1\right] - \frac{V_{\mathrm{oc},T'}}{R_{\mathrm{sh}}} = 0 \tag{1.8}$$

The second new equation (Equation 1.9) compares the γ_{spec} with the γ_{model}. On a solar panel datasheet γ refers to the temperature coefficient of P_{max}, i.e. the percentage by which the max power point changes per degree change in temperature. This works similarly to the previous equation, by adjusting the parameters to their actual conditions given temperature, then finding the max power point of the adjusted curve, and calculating the rate of change. If the parameters are correct, then the γ_{spec} will match the γ_{model} from the datasheet [16].

$$\gamma_{\mathrm{spec}} = \left.\frac{\mathrm{d}P}{\mathrm{d}T}\right|_{\mathrm{mp}} \approx \frac{P|_{T_{\mathrm{cell}}} - P|_{T_{\mathrm{cell,SRC}}}}{P_{\mathrm{mp,SRC}} \cdot (T_{\mathrm{cell}} - T_{\mathrm{cell,SRC}})} \tag{1.9}$$

Informed Initial Guesses

Dobos undertook further research to inform initial guesses for the parameters in the least square solver. For simplicity, the single diode equation was altered to use modified diode ideality (a) as shown in Equation 1.10.

$$I = I_{\mathrm{ph}} - I_s \left[\exp\left(\frac{V + IR_{\mathrm{s}}}{a}\right) - 1\right] - \frac{V + IR_{\mathrm{s}}}{R_{\mathrm{sh}}} \quad \text{where} \quad a = \frac{nN_{\mathrm{s}}kT}{q} \tag{1.10}$$

To begin with the a value was tested for panels of varying technologies (materials e.g. silicon) and number of cells, and a linear regression run to inform most likely a values. The results are shown in Table C.1 in Appendix C. A similar process was used for R_s and R_{sh}, which can be

roughly estimated at the slope of the I-V curve where $V = V_{oc}$.

$$R_{\mathrm{s}} \approx C_{\mathrm{s}} \frac{V_{\mathrm{oc}} - V_{\mathrm{mp}}}{I_{\mathrm{mp}}} \qquad \text{and} \qquad R_{\mathrm{sh}} \approx C_{\mathrm{sh}} \frac{V_{\mathrm{oc}}}{I_{\mathrm{sc}} - I_{\mathrm{mp}}} \tag{1.11}$$

Equation 1.11 was used to inform estimates for R_s and R_{sh}, using coefficients measured from pre-solved panels, and averaged out per technology. The coefficient values can be seen in Table C.2 in Appendix C. Finally, for I_{ph} the estimate of the panel's I_{sc} is recommended and I_s uses Equation 1.12 to get an initial estimate [16].

$$I_{\mathrm{s}} \approx I_{\mathrm{sc}} \exp\left(\frac{-V_{\mathrm{oc}}}{a}\right) \tag{1.12}$$

Validity Checks and Heuristics

The research further states that although a solver may converge, the parameters converged on may be incorrect. If incorrect, then the I-V curve will be inaccurate, and the P_{max} point may be invalid. The paper proposes a series of tests to ensure a valid curve. The first was a set of sanity checks of the parameters as shown in Table C.3 in Appendix C. The second is to ensure that the P_{max} point matches the datasheet value, the third ensures that $I = 0$ at V_{oc} and the last ensures that the derivative of the slope is always negative.
If the parameters found via a convergence pass all these tests, then the shape of the I-V curve is correct, and the parameter values are accepted. If a test or convergence fails, then the solution is rejected. If a solution is rejected, Dobos suggests 3 heuristic methods to update the solvers initial guess, to improve the chances of successful convergence. The first was for amorphous silicon panels. When a panel of this type failed, the initial guess for modified diode ideality would be first decreased by factors of 1.2, 1.4, 1.6, 1.8, 2.0 and 2.2 and then increased by the same factors, retrying at each step. The second heuristic was for multicrystalline silicone panels. If convergence failed, sometimes it could succeed by decreasing I_s and R_{sh} by factors of 100 and 2 respectively from their initial guesses before reattempting the solver.
The final heuristic was proposed to work regardless of technology. This works by increasing the I_{sc} by 1% a maximum of 5 times, which reduces the I-V flatness near the I_{sc} point, and rerunning the solver. This was found to help induce convergence of some modules, while slightly distorting the I-V curve. However for modelling, the P_{max} point is the important part, and this remains mostly unaffected by a higher I_{sc}. Dobos claims a 99.87% successful convergence rate with these heuristics [16].

1.2.5 Artificial Neural Network (ANN) for Parameter Estimation

Due to the least square method required for these parameter extraction methods, slow performance can be assumed. Therefore it is necessary to reduce this complexity. Karatepe suggests the usage of an ANN for this purpose [17]. ANNs are networks made up of input, output and hidden layers, used to take in one set of parameters and estimate another, using the hidden layers for calculation [18].
To train an ANN to accurately predict these parameters would require a dataset, where the values of the parameters of a cell are recorded at different temperatures and irradiances. This

will be split into a training/test set, and the ANN will use the training set to adjust its weights based on the distance between the output value and the actual value. The test set is then used to ensure accuracy [18]. For this purpose, the optimal structure of the ANN needs to be found.

1.3 Panel Mismatch Modelling

1.3.1 Modelling as a Whole Panel Vs. as a Series of Cells

While the single diode model discussed above allows parameter extraction and calculation of a whole panel's I-V curve, it requires the panel to be considered as a single entity, meaning the irradiance and temperature has to be constant across the whole panel. This removes the ability to calculate mismatched shading, which is the goal of this project.
To account for mismatch, the panel level parameters need to be converted to cell level. As discussed in Section 1.1, cells connected in series share the same current, meaning I_{ph} and I_s would have the same value at panel and cell level. Furthermore, panel voltage is the sum of all cell voltages, so the per cell I-V points can be found simply by $(I_{panel}, V_{panel}/N_s)$, meaning all cell parameters are known except the 2 resistances. These can therefore be found via least square optimisation with the single diode equation and known parameters. With individual cell parameters, individual I-V curves can be generated and used to calculate the output of a mismatched panel.

1.3.2 Implicit Trust Region Modelling

Non-Linear Least Squares

Under shading mismatch, the I-V of components of a solar panel can't be modelled individually, due to the structure of solar circuits. Cells being connected in series means that they have to have a constant current, bypass diodes' voltage must be opposite to the voltage of cells connected to them in parallel, and the cells I-V curve needs to fit with its 5 parameters. Therefore, cells need to be modelled in context of all other components. For example, a module with 12 cells and 3 bypass diodes, has 30 unknown variables (15 voltages and 15 currents). To solve these, Karatepe proposes a series of equations to use with the trust region method [17]. These equations are shown in Table 1.1.
The trust region method involves iteratively updating variables to reduce the error of a series of functions. Due to the number of variables and equations, a non-linear least square approach is needed, meaning the results of the equations given the variables need to be collected as residuals, and the variables updated until the residuals are reduced to 0. This solver should take in a panel voltage, and use non linear least square methods to find the best fitting I-Vs for each cell.

Max Power Point Tracking (MPPT)

Rationale of MPPT Finding a panel's P_{max} means finding the V of the panel that gives the highest power output given the I-V point. It would be simple to calculate the whole P-V curve, using the least square solver, to find the highest value. However, this would be

Equation	Explanation
$\sum_{i=1}^{c} V_i - V_{\text{load}} = 0$	States that the total voltage load (output of the solar module) must be equal to the sum of all individual cell voltages.
$\sum_{t=p(i-1)+1}^{ip} V_t + V_{c+d-1+i} = 0$	States that the bypass diode (d) must have the negative voltage equivalent to the sum of all voltages of the solar cells it is connected to in parallel. (for $i = 1, \ldots, d$)
$I_n - I_{n+1} = 0$	The current of all series-connected cells must be identical (Kirchhoff's Current Law). (for $i = 1, \ldots, c$)
$-I_i + I_{\text{ph}}(i) - I_{\text{s}}(i)\left[\exp\left(\frac{q[V_i + I_i R_{\text{s}}^{\text{cell}}(i)]}{n(i)kT(i)}\right) - 1\right] - \frac{V_i + I_i R_{\text{s}}^{\text{cell}}(i)}{R_{\text{p}}^{\text{cell}}(i)} = 0$	Rearranged single diode equation for each cell, determining the current and voltage equilibrium based on the five parameters. (for $i = 1, \ldots, c$)
$-I_{c+(d-1)+i} + I_{\text{sbd}}\left[\exp\left(\frac{q[V_{c+(d-1)+i}]}{n_{bd}kT_{bd}}\right) - 1\right] = 0$	Single diode equation for bypass diodes, using constant parameter values to find the specific current and voltage equilibrium. (for $i = 1, \ldots, d$)

Table 1.1: Module equations for the trust-region module I-V solver.

minimally $\mathcal{O}(i \cdot K \cdot (m \cdot n^2))$, as discussed in Section 1.5, where i is the number of voltages tested. Instead, graph maximum finding algorithms, could be used to improve this process.

Particle Swarm Optimisation (PSO) However, with bypass diodes and mismatched shading, P-V curves tend to have several local maxima, making standard algorithms less effective, as they can get stuck at a local maximum [19]. Instead, algorithms based on Particle Swarm Optimisation (PSO) can be used [19, 20]. PSO aims to solve the continuous variable optimisation problem. It operates by generating several particles, each represented by a position on a graph and velocity, and follows Algorithm 1, to find the optimal particle position [20]. In terms of finding P_{max} from a P-V curve, the particle position would be a panel voltage value. Each particle would generate a P-V point (the higher the power, the better) from this voltage using the I-V least square solver. Equation 1.13 shows how these values are used to update the velocity and particle position, with c1, c2 and w being the cognitive, social and inertia weights used for optimisation. v and x represent the particles velocity and position [20].

$$v_{i+1} = wv_i + C_1(P_{\text{best}} - x_i) + C_2(G_{\text{best}} - x_i) \qquad \text{and} \qquad x_{i+1} = x_i + v_{i+1} \tag{1.13}$$

Deterministic Particle Swarm Optimisation (DPSO) The literature suggests several different algorithms extending PSO, to improve suitability to solar MPPT, as PSO is known to converge prematurely on local maxima under shading mismatch. One widely used method is Deterministic Particle Swarm Optimisation (DPSO), as it reduces the randomness of PSO [19]. In standard PSO the velocity of a particle can't be predicted. This could lead to overshooting

Algorithm 1 Basic PSO Algorithm

Input: Number of iterations, population size
Output: Global optimal position vector

$t = 1$
Initialize particle position vectors and velocity vectors
While $(t \neq t_{\text{Max}})$ **do**:
  Calculate fitness of each particle
  Update local optimal position of particles
  Update global optimal location of the population
  Update position and velocity of particle
  $t = t + 1$
End while
Output global optimal position vector

the global maximum or taking too long to converge [19, 20]. DPSO removes the randomness of PSO by reworking the velocity equation from Equation 1.13 to Equation 1.14, so that there is only one weight (w) to tune for optimisation.

$$v_i^{k+1} = wv_i^k + \left\{ G_{\text{best}} + P_{\text{best},i} - 2x_i^k \right\} \tag{1.14}$$

Reduced Search Space & The Duty Cycle To reduce the search space that DPSO has to search, Mohapatra et al. recommends using duty cycles (D) in place of voltages [19]. Duty cycles control how often a solar panels converter is active, from 0 to 1. The relationship between panels voltage (V_{in}), converters output (V_{out}) and duty cycle can be seen in Equation 1.15 [21].

$$M(D) = \frac{1}{1-D} = \frac{V_{\text{out}}}{V_{\text{in}}} \qquad \therefore \qquad V_{\text{in}} = V_{\text{out}} \cdot (1 - D) \tag{1.15}$$

Mohapatra et al. recommends a reduced range of duty cycles. The P_{max} point will never fall outside the range of duty cycles calculated from Equation 1.16 [19], where ηbb is the converters efficiency, $R_{\text{L min/max}}$ are the minimum and maximum values for the converters voltage and $R_{\text{PV min/max}}$ are the reflective impedances of the converter [19]. Reflective impedance is the ratio of voltage to current (V/I). It is similar to resistance, as it increases as irradiance and temperature move away from standard operating conditions. To model as an ideal panel, can assume that the converter has 100% efficiency, and the same variables (e.g. same min/max voltage) as the panel.

$$D_{\text{min}} = \frac{\sqrt{\eta_{\text{bb}} R_{L,\text{min}}}}{\sqrt{R_{PV,\text{max}}} + \sqrt{\eta_{\text{bb}} R_{L,\text{min}}}} \qquad \text{and} \qquad D_{\text{max}} = \frac{\sqrt{\eta_{\text{bb}} R_{L,\text{max}}}}{\sqrt{R_{PV,\text{min}}} + \sqrt{\eta_{\text{bb}} R_{L,\text{max}}}} \tag{1.16}$$

Hybrid DPSO & P&O Tracker While the DPSO improvements should increase the speed of the P_{max} search, it is still likely to be slow due to having to search the graph globally. It is recommended to combine this method with a local tracker such as perturb and observe (P&O) [19], which works by stepping in a direction and testing for change. If the power increases, it continues stepping in that direction, if not, the direction swaps. This should be more efficient, but is also likely to get stuck at local maxima. Therefore a combined approach using DPSO to

find the global maximum under drastic power changes, and P&O to maintain this maximum, would be optimal [22].

1.3.3 Explicit I-V Calculations & Avalanche Breakdown

Explicit Single Diode Equation

The format of the single diode equation discussed thus far requires implicit solving to find voltage from current. Bishop proposes a reformatting, which separates the equation into two explicit equations [23]. Equation 1.17, shows how I can be taken out of the exponential of the single diode equation and replaced with the diode voltage (V_j). The right hand equation can then be used to find the terminal voltage (V).

$$I = I_{\text{ph}} - I_{\text{s}} \left[\exp\left(\frac{qV_j}{nkT} \right) - 1 \right] - I_{\text{shunt}} \qquad \text{and} \qquad V = V_j - IR_{\text{s}} \tag{1.17}$$

In addition, the right hand term of the diode equation ($\frac{V+IR_s}{R_{sh}}$) is replaced with (I_{shunt}), to remove I from the right hand side of the equation. When looking at Figure 1.1, in the first quadrant, we can see that as the voltage becomes increasingly negative, eventually it breaks down, and the current begins to rapidly rise, causing an exponential shaped curve. This is known as the avalanche breakdown [23]. Bishop proposes Equation 1.18 to use the the breakdown voltage (V_{br}), the avalanche fraction (a) and the avalanche exponent (m) to model this breakdown explicitly in the I-V curve [23].

$$I_{\text{shunt}} = \frac{V_j}{R_{\text{sh}}} \left(1 + a \left(1 - \frac{V_j}{V_{\text{br}}} \right)^{-m} \right) \tag{1.18}$$

Algorithm 2 Current-First Approach for Module P-V Curve

Input: Module class with list of cell class at realistic conditions
Output: P-V curve

```
Get max cell I_ph from cell list
Define sweep from 0 to max I_ph
Divide cell list into substrings per bypass diode
For substring in substrings:
  For cell in substring:
    Explicitly calculate voltage sweep from current sweep
    Add to overall substring voltage
  Clamp negative voltage to bypass voltage
  Add substring voltage to module voltage
Clean negative module voltages
Return voltage and power
```

Explicit Solver

Using these equations, a solver could explicitly calculate V from I for each cell, and summate the voltages to first find a substring, then a panel I-V curve, accounting for mismatch. The full algorithm can be seen in Algorithm 2. This method is assumed to be faster, due to using

purely explicit equations, whereas previous methods required trust region methods and iteration. The hypothetical complexity would be $\mathcal{O}(c \cdot i)$, where c is the number of cells and i the number of currents to test. However, it is theorized to be less accurate, due to not being as heavily constrained by physical realities as the implicit model.

1.4 Graphical Modelling and Irradiance

There are 3 types of irradiance to model on a panel: diffuse, reflected and direct beam. Diffuse and reflected have minimal impact on power, so a simple model should only use direct beam [24]. Direct beam radiation is the irradiance directly from the source to the object and depends on the angle between the objects normal and the source. To calculate the irradiance on a solar panel, Equation 1.19 is used, where $\hat{\mathbf{n}}$ is the unit normal vector of the cell surface and $\hat{\mathbf{s}}$ is the unit vector pointing in the direction of the light source, both having been normalised. According to Lambert's Cosine Law, their dot product is then used to scale the maximum direct irradiance (G_{max}) to determine the effective irradiance (G_{eff}) received [25].

$$\cos(\theta) = \hat{\mathbf{n}} \cdot \hat{\mathbf{s}} \qquad G_{\text{eff}} = G_{\text{max}} \cdot \cos(\theta) \tag{1.19}$$

To take shading into account, the literature suggests ray tracing methods [24], where a ray would be sent out from the light source towards a point, and if the light intersects with a triangle, then that triangle would be unshaded. However, other literature suggests reverse ray tracing, where the ray originates from the point, may be more suitable. If this ray intersects with another object before reaching the light source, then the point is shaded [26]. A further improvement of this model would be to account for penumbra (partial shading) as well as umbra. With a single ray, a cell can be either shaded or unshaded, therefore, it is suggested that an accurate graphical model would use distributed ray casting [26] to identify the edge of shading, where penumbra is likely.
Distributed ray casting means instead of firing a ray from a single point on the cell to test if it's shaded, which would limit to either shaded/unshaded, the algorithm randomly samples from possible points, to cast the ray from [26]. The intensity of the shade can be extrapolated from the proportion of shaded points in a set area.

1.5 Existing Models & Limitations

PVLibs

`PVLibs` is a python library developed to model the power output of a panel or string of panels, using the `singlediode` function. [27]. `PVLibs` also contains a comprehensive CEC database, which holds technical information on a range of solar modules. The main limitation of this library is that it treats each panel as a single object, i.e. a panel is either shaded or unshaded, meaning it can't account for mismatch. In addition, to calculate shading, `singlediode` requires the 5 parameters at operational conditions. These can be calculated by the `calcparams_desoto` method, but this requires the reference 5 parameters. While these are stored in `PVLibs` database, they are not included in datasheets, meaning only panels in the

database can be calculated.

PVMismatch

`PVMismatch` is another python library, developed to explicitly calculate the I-V curves of a panel under mismatched conditions [28]. It allows a panel to be initiated, and the function `setSuns` to take in an irradiance array, to adjust each cell to the parameters at its own shading level. From this, the variables `Vsys` and `Psys` (the panels P-V) will be adjusted to account for the mismatched shade. The limitation of this library is that it can't be used independently. It uses the 7 parameter model, and requires these parameters to be pre-calculated, if a user wants to model a non-standard panel, as it has no way to calculate reference cell parameters itself. Finally, neither `PVLibs` nor `PVMismatch` have methods to calculate shade. `PVLibs` can simply calculate irradiance based on time of day and location, but can't account for, as an example, shading caused by a tree. `PVMismatch` requires the irradiance array to be calculated for it.

SciPy & Complexity

The python library `SciPy` offers two different functions that will be of use to this project: `least_squares` and `fsolve`. These functions are preferable to developing equivalent functions, as they are wrappers around compiled languages such as `C`, meaning a faster execution speed than `Python` loops [29].

least_squares This function implements the non-linear least squares method required to extract the 5 parameters and generate a panels P-V curve as discussed in Section 1.3.2. It operates with a series of functions $f_i(\mathbf{X})$, known as residuals, and aims to find the $\mathbf{X}$ values to minimise the sum of the squares of these functions [29]. This uses a $m \times n$ matrix of partial derivatives, known as a Jacobian matrix (J), with each partial derivative demonstrating how sensitive the residual is to a change in the variable [29, 14].
To inform the variables step direction and magnitude, the solver calculates $J^T J$ to approximate the 2nd derivative and the gradient ($J^T r$). From this the solver uses Equation 1.20, where r are the residuals and λI is a dampening function [14], to solve for Δx, which is used to iterate the variables. Based on the iterative process, and the required $m \times n$ matrix multiplication, a complexity of $\mathcal{O}(K(m \cdot n^2))$, where K is the number of iterations, can be assumed.

$$(J^T J + \lambda I)\Delta x = -J^T r \qquad x^{k+1} = x^k + \Delta x \tag{1.20}$$

fsolve This function works to find the root of non linear equations, meaning it can be used to calculate I from V (or vice versa) using the single diode equation. This involves iterating a variable or array of variables, via Newton's method, as discussed in Section 1.2.2. Sorensen et al. explains that this method operates on systems where the number of variables equals the number of equations. Therefore it calculates an $n \times n$ Jacobian matrix, where n is the number of variables. To find the step direction, the method must then solve n non linear equations [9]. Due to calculating the $n \times n$ matrix and non linear solving at each step, we can assume a complexity of $\mathcal{O}(K(n^3))$, where K is the number of iterations.

Chapter 2

Methodology

2.1 Design of Proposed Solution

2.1.1 The Research Gap & Project Overview

As discussed in the limitations section, currently there is an incomplete chain to model solar panel mismatch. To calculate using `PVMismatch` cell level parameters are required, which can't be generated using either library. In addition, there is no way to model how shade will fall on a panel. Therefore, this project aims to develop a complete chain to generate I-V curves, based on realistic irradiance data, and openly available datasheet information.

2.1.2 Project Management

The overall goal of this project, is to investigate the viability and performance of different methods, in order to model the P_{max} output of solar panels. As such, it will be run as an agile project. Each step will be individually implemented, validated, assessed for accuracy, and refactored if required.

2.1.3 Proposed Development Pipeline

To maintain some rigidity and organisation, the development cycle was divided into 3 major pipelines. The first would be the parameter prediction pipeline, which would involve implementing and testing the various methods to extract panel parameters, converting them to cell level, before building, optimising and testing the ANN, using the `Sci-Kit Learn` python library [30]. `Sci-Kit Learn` was chosen due to ease of use, quick performance and comprehensive documentation.

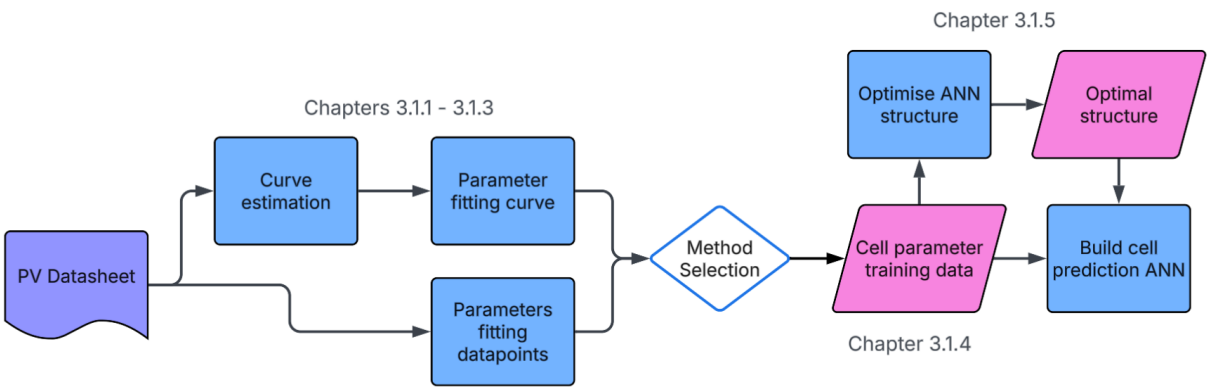


Figure 2.1: The calculation pipeline for predicting cell parameters at various conditions based on a photovoltaic modules datasheet

The second pipeline involves developing the two whole panel solvers, using the results from the previous pipeline. This means developing the implicit and explicit solvers as discussed in Section 1.3, alongside the MPPT modules, and comparing the 2 for performance and accuracy.

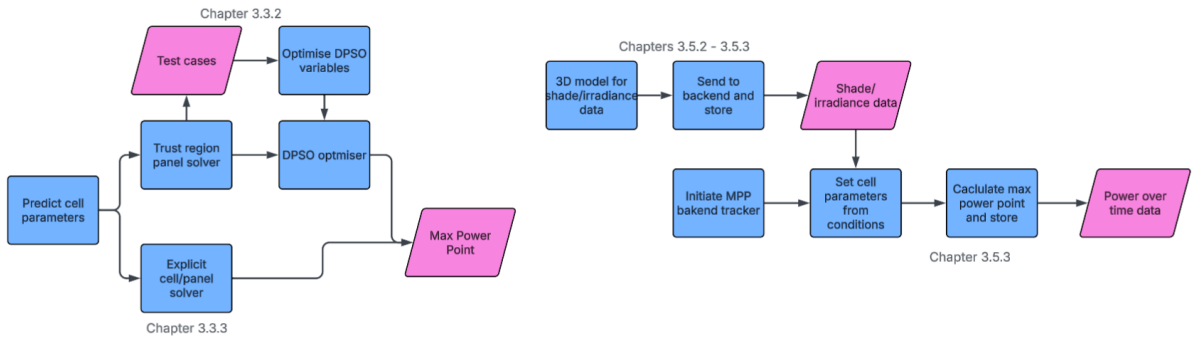


Figure 2.2: The pipeline to develop and calculate the max power point tracking system (left) and the pipeline to incorporate this with the graphical model to calculate power over time (right)

The final pipeline is the data gathering pipeline, which focuses on developing the ray tracing methods and the client-server architecture to send the irradiance data to the P_{max} solvers.

2.1.4 System Design

To manage the complexity of this project, the architecture will be divided into 2 parts: a Unity (C#) frontend handling the rendering and ray casting, and a Python backend managing cell parameter prediction, the panel power-voltage (P-V) solvers and the max power point tracking. This separation will ensure that complex computation doesn't impact the graphical model.

Computational Backend (Python)

The backend will be where the core computation takes place. The physical hardware will be modelled as a hierarchy, with the panel class at the root, which is made up of a list of cell objects, allowing module level parameters to be divided down to the cell level under mismatch. Here, the cell class will interact with an ANN, to predict the 5 parameters.
The panel class will interact with the 2 different methodologies of the implicit and explicit panel solvers, which will be implemented and performances compared. Alongside this, an MPPT (tracker) class will create the base structure for the 3 different methods tested, PSO, DPSO and hybrid. This tracker will interact with the implicit solver of the module class.

Graphical Frontend (Unity/C#)

The frontend architecture will focus on efficiently rendering and calculating irradiance data on the panel. Unity will be used, due to its inbuilt lighting and ray casting methods, and ease of setting up objects and light sources. A ray casting class will handle the reverse ray-casting and random sampling logic for accurate shade generation. The panel manager will then aggregate the cell data into a float array, allowing network transmission. In order to simulate a realistic sun, Unity will use a clock to drive a light controller, simulating a day's trajectory.

Inter-Process Communication

A UDP (User Datagram Protocol) client-server model will bridge these two sections, transmitting the irradiance array from Unity to a python UDP server. UDP will be used over TCP (Transmission Control Protocol), as it has reduced overhead, and does not require a

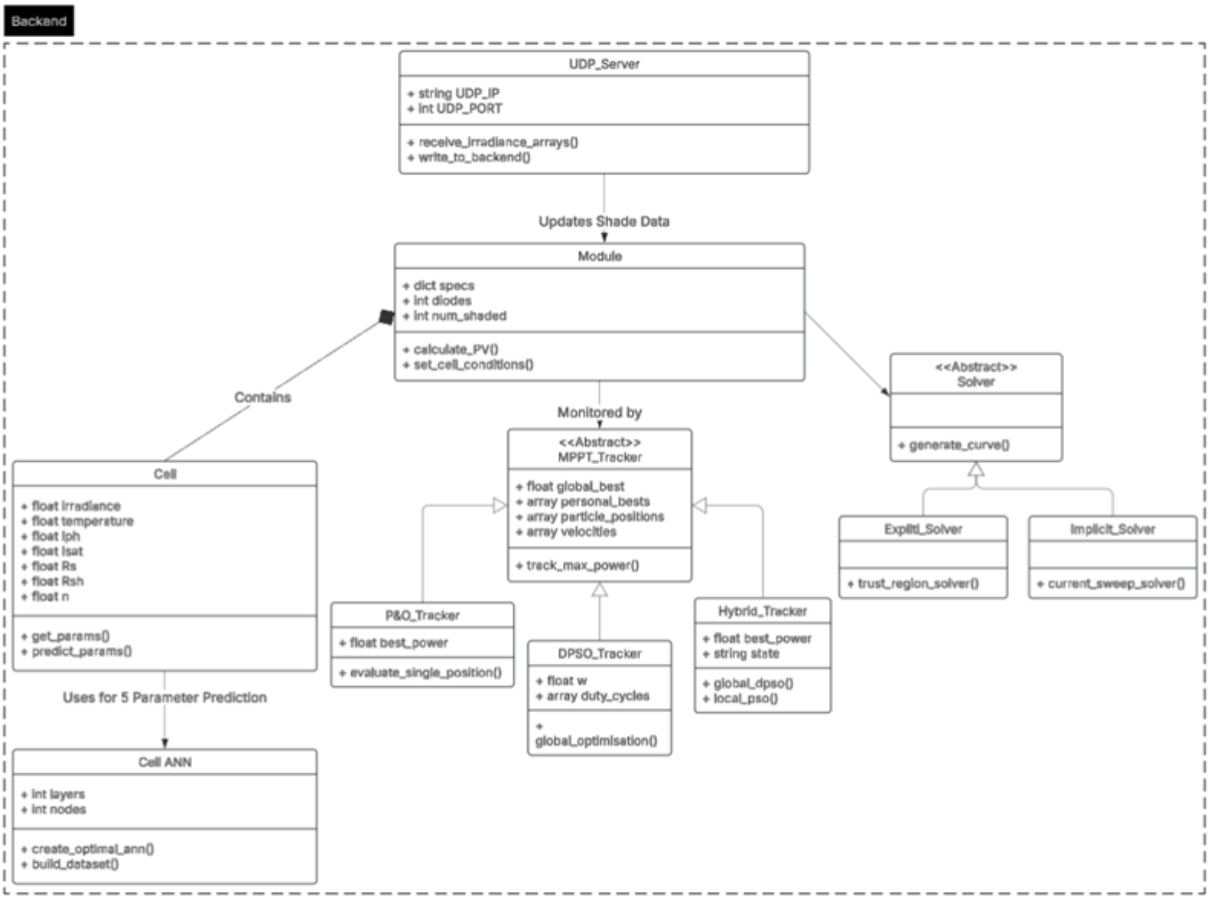


Figure 2.3: A UML class diagram showing the ideal structure of the python backend to capture the max power point of a panel from irradiance data

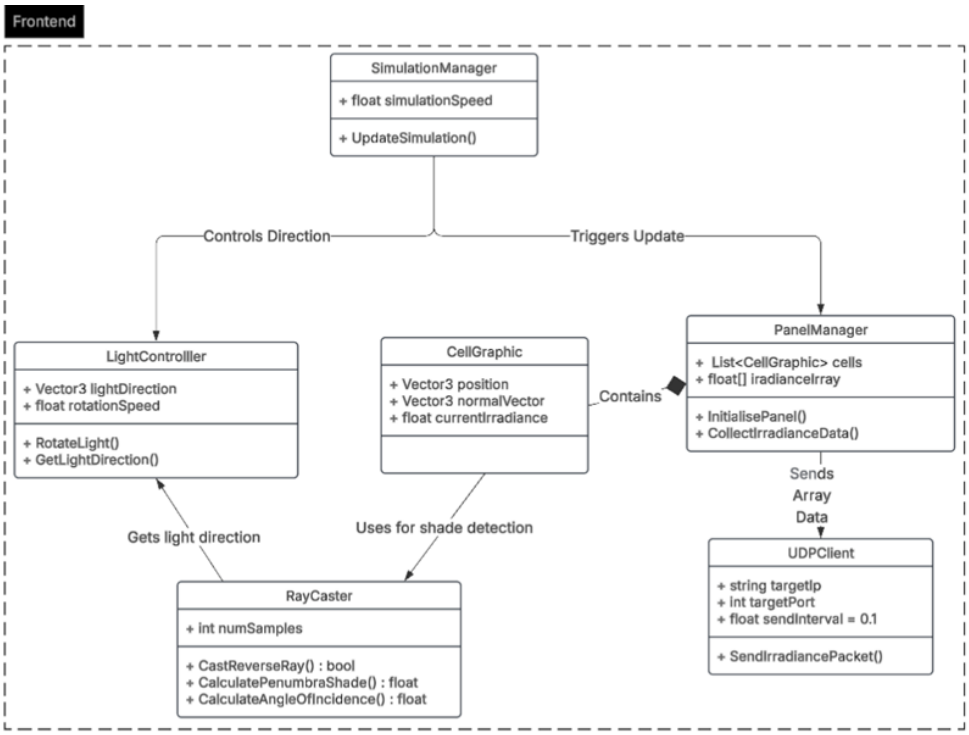


Figure 2.4: A UML class diagram showing the ideal structure used to graphically capture the irradiance data on a panel given a light source and shading

constant connection, allowing for smoother interaction and less computational expense from TCPs handshake overhead. This data will be written to a CSV file, to prevent packet buildup caused by the slower computational backend.

2.2 Validation Plan & Version Control

2.2.1 Version Control

To allow iterative development, matching the discussed pipelines, Git version control will be used. Issues will be set up to work as guidelines for different targets in the pipeline. Branching will be implemented, to isolate experimental changes to the backend methods. Each pipeline will be managed as a sprint, with development, testing and validation performed at each stage.

2.2.2 Validation Plan

Pipeline 1 Validation - Predicting Cell Parameters

In order to first validate the panel parameters, the `PVLibs` `calcparams_desoto` function can be used. By testing a panel that is both in the `PVLibs` CEC database, and we have the necessary datasheet conditions for, we can ensure that the extraction is accurate, and extrapolate that this would work for panels not in the database. `PVLibs` does not allow cell level prediction, so to validate these results, we will look for correctly formed I-V curves, with $I_{cell} = I_{panel}$, and $V_{cell} = V_{panel}/N_s$. For the ANN, we can simply validate its results against the extraction method.

Pipeline 2 Validation - Module I-V & $P_{\mathbf{max}}$

To validate the module solvers, a series of tests will be run to compare the shape of their P-V curves, against a custom PVMismatch panel. This custom panel will have to be built using a series of cells with the parameters calculated from the previous pipeline, in order to match our results. From here, the MPPT can be tested against the peak from a whole P-V graph.

Pipeline 3 Validation - The Graphical Model and Irradiance Data

This section will be more difficult to validate. With a moving light source, we can know what is expected, i.e. day/night cycle and a power drop when shaded, but have no empirical values for what irradiance should be. Therefore, the power over time curves will be assessed qualitatively, and snapshots taken from the graphical method, to ensure that P-V graphs at individual times are working as expected.

2.2.3 Unit Testing & Continuous Integration (CI) Pipeline

At each stage, validation will be performed using a series of unit tests written with `PyTest` [31]. These will be grouped in a folder called `regression_tests`, and implemented as part of a CI pipeline, to be run for each pull request, ensuring that the software still operates as expected.

Chapter 3

Implementation and Validation

3.1 Pipeline 1 - Predicting Cell Parameters

3.1.1 Reference Parameter Extraction

The first target was to develop the methods to extract the reference 5 parameters (at STC) at panel level. The panel used in this section was `Canadian Solar Inc. CS1U-405MS`, as its datasheet was publicly available [32], and it was also present in `PVLibs` database.

Condition	Explanation	Value
V_{oc}	Open circuit voltage, when current is 0	53.5 V
I_{sc}	Short circuit current, when voltage is 0	9.65 A
V_{mp}	Voltage at max power point	44.3 V
I_{mp}	Current at max power point	9.16 A

Table 3.1: Datasheet conditions for the Canadian Solar Inc. CS1U-405MS used for development and testing.

Simulated Curve

The first method implemented was discussed in Section 1.2.3, and uses Equation 1.2 to approximate an I_{TA} value for each voltage between 0 and V_{oc}. The curve was compared to `PVLibs i_from_v` and shown in Figure 3.1, which demonstrates a near perfect match.
As discussed, a solver was written to use `least_squares` to fit the 5 parameters to this curve, using Equation 1.4 with the (I_{TA}, V) pairs as the residual function. This solver was too unstable, pushing I_{ph}, R_{sh} and n to their minimums, and I_s and R_s to their maximums. To attempt to stabilise the solver, the number of variables was reduced, removing I_s, as it was furthest from the correct value. Instead it was passed as a constant the from `PVLibs` database.

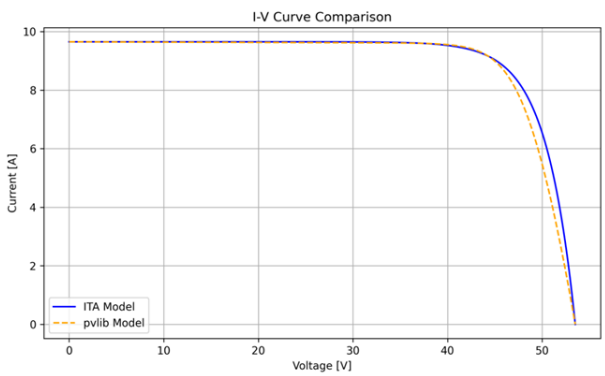


Figure 3.1: A graph comparing the `PVLibs` generated I-V curve, and the approximated I_{TA} curve

Point Fitting & the $P_{\mathbf{max}}$ Derivative

The second method used the same solver, except the points from the simulated curve were replaced by the known points on the datasheet (e.g. $(0, V_{oc})$), removing the ambiguity of the simulated curve. In addition, Equation 1.5 was added to the residual, to further constrain the solver. Again this struggled with convergence with a variable I_s, so this was taken as constant.

3.1.2 Operational Parameter Calculations

For the project, the parameters needed to be known at operational conditions, where irradiance and temperature vary. For this, a function was written to take in the reference parameters calculated above, an irradiance and temperature value, and use the equations from Section 1.2.3 to return the operational parameters. For simplicity, air mass (m) and air mass reference (m_{ref}), are taken as constants, such that $(m/m_{ref}) = 1$ and $E_{\text{g, ref}} = 1.21 \times 10^{-19}$. This assumes the panel is silicon, and that the light doesn't have to travel to reach it.

3.1.3 Heuristic Method

The final solver implemented used the heuristic method discussed in Section 1.2.4. This method is likely to be more accurate due to the validity checks and additional known variables. It requires additional datasheet values: the technology type of the panel and the temperature coefficients of V_{oc}, I_{sc} and P_{max} (β_{oc}, α_{sc} and γ respectively).

Improved Residual

The updated residuals can be seen in Equation 3.1, where X are the target variables, and F are the equations. The first 4 equations are the point fitting and derivative equations from the previous section. The fifth uses the operational parameters function, and the temperature coefficients, to ensure that the parameters match the adjusted single diode equation at +5°C. The sixth works similarly, by finding γ_{model}. To do this, the 5 parameters were iterated every 1°C between −10°C and 50°C, and the P_{max} found at each temperature. The average of the slope would then be found, which would demonstrate how much the P_{max} changes per °C. In the residual, this would be tested against the datasheet P_{max} coefficient, meaning the closer the calculated value is to the actual value, the lower the residual.

$$\mathbf{x} = \begin{bmatrix} a \\ I_{ph} \\ I_s \\ R_s \\ R_{\text{sh}} \\ \text{Adjust} \end{bmatrix} \qquad \mathbf{F} = \begin{bmatrix} I_{ph} - I_s\left(\exp\left[\frac{I_{\text{sc}}R_s}{a}\right] - 1\right) - \frac{I_{\text{sc}}R_s}{R_{\text{sh}}} - I_{\text{sc}} \\ I_{ph} - I_s\left(\exp\left[\frac{V_{\text{oc}}}{a}\right] - 1\right) - \frac{V_{\text{oc}}}{R_{\text{sh}}} \\ I_{ph} - I_s\left(\exp\left[\frac{V_{\text{mp}}+I_{\text{mp}}R_s}{a}\right] - 1\right) - \frac{V_{\text{mp}}+I_{\text{mp}}R_s}{R_{\text{sh}}} - I_{\text{mp}} \\ I_{\text{mp}} - V_{\text{mp}}\left[\frac{\frac{I_s}{a}\exp\left[\frac{V_{\text{mp}}+I_{\text{mp}}R_s}{a}\right] + \frac{1}{R_{\text{sh}}}}{1+\frac{I_sR_s}{a}\exp\left[\frac{V_{\text{mp}}+I_{\text{mp}}R_s}{a}\right] + \frac{R_s}{R_{\text{sh}}}}\right] \\ (I_{ph} + \alpha_{\text{sc}} \cdot (1 - \text{Adjust}/100) \cdot \Delta T) \ldots \\ \gamma_{\text{spec}} - \gamma_{\text{model}} \end{bmatrix} \tag{3.1}$$

Convergence Checks

To ensure that the parameters were converging accurately, the boundary and I-V curve checks [16] were implemented, meaning that the solver would reject any values that failed these tests.

This system was run with a subset of the panels in `PVLibs` CEC database, and returned a 56.94% convergence rate, demonstrating unreliability.

Heuristic Implementation

To increase the convergence rate, the heuristics were implemented. If the solver failed, these heuristics would update the initial guess by the discussed values and rerun the solver. With the first two technology specific heuristic (adjusting modified diode ideality of amorphous silicon and I_s and R_{sh} of multicrsytalline silicon), the convergence rate remained at 57%.
The final heuristic updated the I_{sc} for all failed panels by 1%, before repeating the solver, up to 5 times. This drastically increased the convergence rate to 96.08% for the smaller dataset and further increased to 97.3% with a larger dataset. The inference is on the full dataset, the solver would achieve the 99.87% rate found by [16].

3.1.4 The Single Cell Class

For the mismatch solvers, the parameters are required to be converted from panel level to cell level. To do this a `Cell` class was written, which has a set of parameters, an irradiance and temperature, and interacts with the parameter extraction methods. To get operational parameters of the cell, the process works as follows:

- Extract operational parameters from solvers.
- Use parameters to generate an I-V curve.
- Divide the voltages by number of cells to get a per cell I-V curve.
- Use the known parameters, I-V points, and a least square method to extract R_s and R_{sh}.

This will result in a `Cell` object being able to calculate its parameters when given a temperature and irradiance.

3.1.5 Implementing the ANN for Cell Parameter Prediction

Implementation and Optimisation

Due to the previous method requiring a minimum of two least square solvers to calculate the panel level parameters, and to extract the cell parameters from them, this method likely has poor performance. The hypothesis was that an ANN could reduce the calculation time, with a minimal dataset, while still producing accurate estimations for all temperatures and irradiances. A minimal dataset was built, every $100W/m^2$ between 100-1100 and every 5°C between 20-80 ($1100W/m^2$ and 80°C are unrealistic values included to handle edge cases). For the ANN model `MLPRegressor` from `Sci-Kit Learn` was used [30], due to its rapid training and prediction. An optimiser was written using `Optuna` [33], which would create an ANN with a certain number of nodes/hidden layers, before training it on the dataset, and using it to estimate 20 cells at random conditions. The optimiser would calculate the RMSE (Root Mean Square Error) as distance from the actually calculated parameters, the R^2 of the model and the time taken for prediction. `Optuna` was chosen, as it allowed efficient/parallelised running of individual trials, and would return the best structure for the chosen values.

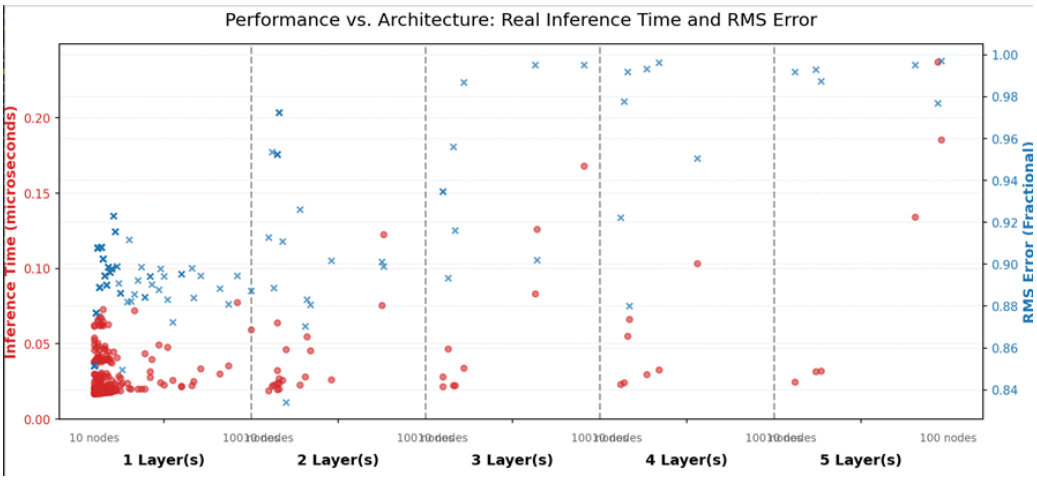


Figure 3.2: A graph showing ANN structural optimisation

After the first round of optimisation, the lowest RMSE was 2, which is high enough to impact the P_{max}. To attempt to reduce the RMSE to a more reasonable <1, two adjustments were made. The first was to remove Rs from the ANN. Rs is kept constant at varying conditions, meaning it didn't need to be recalculated, and reducing the number of values the ANN is required to predict should increase accuracy. The second measure was to reduce the extremity of the I_s values. These were tiny (-2e-13 at STC), meaning that the ANN was forced to weight I_s more than the rest of the parameters. By taking the log_{10} of the values, the ANN could adjust the weights used to better balance the rest of the parameters. The optimiser found an improved RMSE and optimal structure of 2 hidden layers and 33 nodes, as shown by Figure 3.2.

Cell Class Adjustments

The `Cell` class was adjusted to initiate a class level ANN, and run the parameter extraction solver once to find the constant R_s. After that, the prediction process involved calling the ANN with an irradiance and temperature to find the operational cell parameters.

3.2 Pipeline 1 Validation & Performance

3.2.1 Parameter Extraction Method

For the ANN to work properly, the dataset needed to be built accurately. Based on the 3 methods implemented, the heuristic method was chosen. The simulated curve method failed to converge at all and the point fitting method converged, but only with I_s as a constant, limiting its operation to panels in the `PVLibs` database. Due to it's 97% convergence rate and low RMSE from `PVLibs` data, as demonstrated by Figure 3.3, the heuristic method is ideal.

3.2.2 Single Cell Validation

To prove our method of extracting cell level parameters from panel level parameters works, a series of tests were run. Various cell I-V curves were generated using `fsolve` with the single diode equation as the residual, to estimate I from V, as demonstrated in Figure 3.4. We can tell that these curves are valid, due to the cells' V_{oc} sitting at ~0.6V, the expected value for most

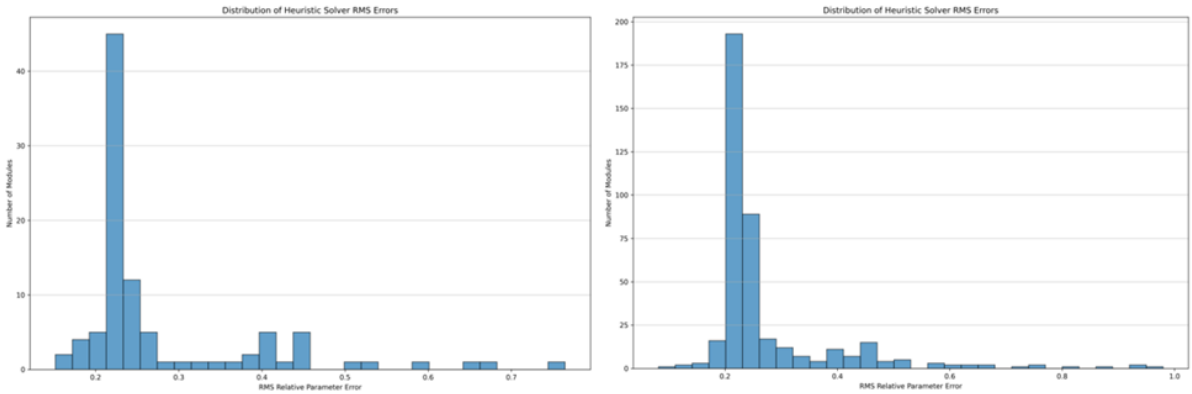


Figure 3.3: Histograms showing the distribution of RMSE of the heuristic solver with a smaller dataset (left) and a larger dataset (right)

solar cells. The curves further demonstrate that as the temperature increases, the I_{sc} increases, and the V_{oc} decreases, which matches their temperature coefficients. Finally, we can see that the current generated drops as the irradiance drops, which follows the lower I_{ph} when shaded.

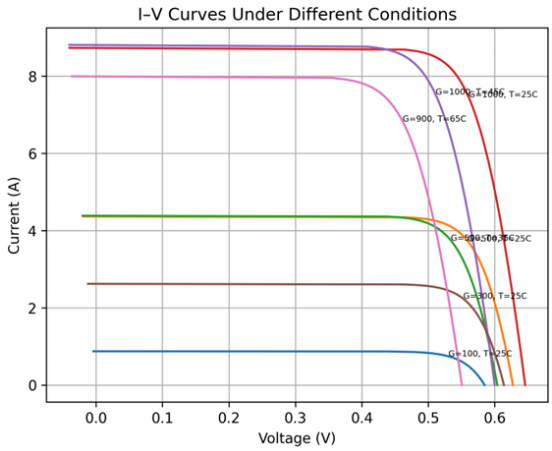


Figure 3.4: A graph showing cell level I-V curves at various cell temperatures and irradiances

3.2.3 Prediction Profiling

The aim of implementing the ANN was to reduce the computational expense of predicting cells. As the ANN is already proven to predict cells with a low error, its viability depends on its performance. To this end, a profiling script was created, which would compare the performance of both the ANN and the heuristic method, when predicting 50 cells at random temperatures and irradiances. As expected, the heuristic method took 25 seconds or half a second per cell, while the ANN only took 0.71 seconds for all 50 cells, a 97% improvement.

3.3 Pipeline 2 - Panel Mismatch Solvers

3.3.1 The Panel Class

The goal of this pipeline was to aggregate cells into substrings and simulate the performance of a whole panel under mismatched conditions. For this the `Module` class was written. This would

initiate `cell_list`, which aggregated a series of `Cell` objects, alongside variable `d`, which controlled the number of bypass diodes and substrings (1 diode per substring). Finally the function `set_cell_conditions`, would take in a array of temperature and irradiance values, which would be used to set the cell conditions and parameters.

3.3.2 Implicit Modelling

Modelling the Panel P-V Curve

Following the implicit method discussed in Section 1.3.2, a non-linear least square solver was implemented. The equations for the residual are shown in Table 1.1, with the goal being that the solver takes in a target voltage (V_{load}) for the panel, and finds the voltages of all cells that summate to it. This needs to be balanced by the physical constraints modelled in the residual equations. However, this method found inconsistencies. For example, in the first substring the cells would be at 8A and then the second would be at 0.5A. This was due to there being no restrictions on current consistency between substrings, so the solver found a stable but incorrect solution. To fix this Equation 3.2 and variable I_{panel} was used to ensure current wasn't lost between substrings.

$$I_{\text{panel}} = I_{\text{cell}} + I_{\text{bypass diode}} \tag{3.2}$$

The I-V curve solver `calculate_iv` was written to iterate across all voltages from 0 to the panel's V_{oc}. At each voltage, the solver would use `least_squares` from `scipy.optimize` [29], to find the I_{panel} value that would allow this target voltage. This would create the I-V graph and, by calculating the product of each point, the P-V graph.

MPPT Implementation

Using the above method, P_{max} can be found from the max point of the whole graph. However, as previously mentioned, a non-linear least square solver is likely to be $\mathcal{O}(K(m \cdot n^2))$, where K is the number of iterations, m is the number of equations, and n is the number of unknowns. Therefore, the `MPPT` class was written to minimise the number of solver calls.

Standard PSO Initially, the `MPPT` class used the standard PSO method, initiating particles at voltages between 0 and V_{oc}. Each particle would call the `PSO_method` from the `Module` class, which ran the solver for a single voltage value, and returned the corresponding power. Based on this, the particles would update their velocities and positions using Equation 1.13, following Algorithm 1 till convergence. Convergence was set to when the P_{max} value stalls for several iterations, or particles become too close to be distinguishable.

DPSO The `DPSO_MPPT` class was written to extend and improve `MPPT`. The main change was to change particle position from voltage to duty cycle. This allowed the search space to be cut down using Equation 1.16. This meant modelling an ideal single panel converter using the values shown in Table 3.2.
This change meant that the `DPSO_MPPT` class had to convert each particles duty cycle position to a voltage by Equation 1.15 before it could be passed into a solver. Additional improvements

Variable	Description	Value
$R_{\text{L, max}}$	Maximum converter voltage	Panel's V_{oc}
$R_{\text{L, min}}$	Minimum converter voltage	0
$R_{\text{PV, max}}$	Maximum reflective impedance	Panel's V_{mp}/I_{mp} at STC
$R_{\text{PV, min}}$	Minimum reflective impedance	Panel's V_{mp}/I_{mp} at WTC
ηbb	Converter Efficiency	1
V_{out}	converters output voltage	Panel's V_{oc}

Table 3.2: Ideal converter values.

included clipping velocities to a min/max to avoid overshooting, and replacing the velocity update step with Equation 1.14 to reduce the number of parameters to optimise.

Hyper-Parameter Optimisation Both of these methods needed hyper-parameter optimsation, in order to find their peak performance. For PSO, this meant optimising the 3 weights and particle count, and for DPSO, the particle count, inertia weight and maximum velocity. This was done by creating several test scenarios, and testing a series of different parameter values to find the values that caused the trackers to converge the fastest, without a reduction in accuracy. The full results can be seen in Appendix C.1.2.

Hybrid DPSO & P&O Tracker To reduce the latency of DPSO, the hybrid DPSO and perturb and observe (P&O) tracker was implemented. The idea was that as the DPSO algorithm solver is likely to perform poorly, it should only be used if a large change in power is detected. Therefore, the `DPSO_MPPT` was given two potential states: `"local"` or `"global"`. In `"global"`, DPSO would run, and in `"local"`, P&O would run. The tracker would start in `"global"` to find the overall peak, before switching to `"local"`, to stay balanced at P_{max} under minimal shading changes. If the power change is drastic between 2 steps in `"local"`, then the state will swap to `"global"`, to find the global peak again.

3.3.3 Explicit Modelling

The `refactored_iv` function from the `Module` class follows the algorithm described in Section 1.3.3 and uses the explicit Equation 1.17 for the cell I-V calculations. For this equation, Bishop recommends values for V_{br}, a and m, and a different set of these values can also be found in the `PVMismatch` library [28]. The values from `PVMismatch` were chosen, as it is more recent, and therefore the values better suited to modern solar cells. This method uses `numpy` arrays in order to vectorise calculation to improve performance [34].
To begin with, the function generated a sweep of currents to test from 0 to the max I_{ph} of all cells in the modules `cell_list`, before dividing the `cell_list` into substrings. Each substring will be iterated through, summating the corresponding cell voltage sweeps to a substring level. From here any current that produces a negative substring voltage will have the value clamped to -0.5 (the bypass diode activation current). After clamping negative values, summating each substrings voltage together will produce whole panel I-V curves.
Due to the use of `numpy` vectorising the I to V calculation, the complexity of this method should only be $\mathcal{O}(n)$, where n is the number of cells in the panel, however it is assumed to be

less accurate due to artificial bypass diode activation, and less accurate explicit equations.

3.4 Pipeline 2 - Validation

3.4.1 Implicit & Explicit Curve Validation

To begin with, the 2 different methods were compared against `PVMismatch`. To do this, the cell reference parameters were calculated, and used to set up a custom panel, before shading patterns were applied to both the `Module` class and the custom `PVMismatch` panel. The `calculate_iv` (green - implicit) and `refactored_iv` (red - explicit) functions were called and the results collated and compared in Figure 3.5. Contrary to the hypothesis, the explicit method tends to match the P-V curves from `PVMismatch` much better. While the implicit method more closely matches the P_{max} point in 6/8 graphs, the explicit curve matches an approximation of this point on all 8.

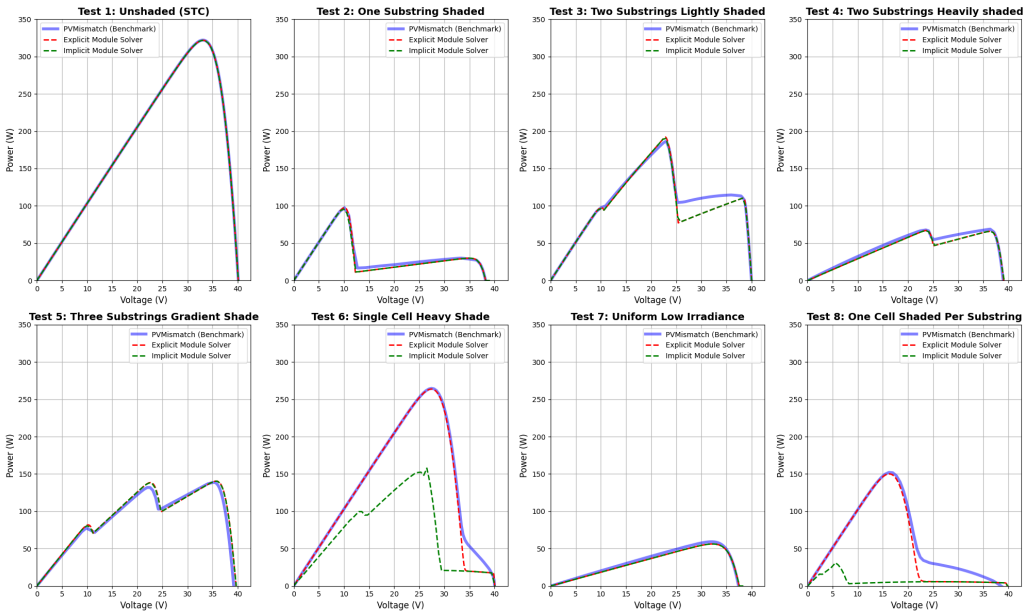


Figure 3.5: A series of graphs showing the accuracy of the implicit and explicit models against `PVMismatch`.

3.4.2 MPPT Validation

DPSO & PSO

In order to validate PSO and DPSO, a test was set up using the implicit model to create a full I-V curve under partially shaded conditions. The 2 methods were then profiled, to see how long they took to find the P_{max}. The results are shown in Figure 3.6. PSO took 24 seconds, and got stuck at a local maximum, as the literature suggested it might. DPSO took only 7 seconds a minimum 2x improvement on generating the whole graph, and found the global maximum.

Hybrid DPSO & P&O

To analyse the performance of the hybrid tracker method, it was initiated, and the shade on the panel it was tracking updated over time for 100 steps. At each step, the whole P-V graph

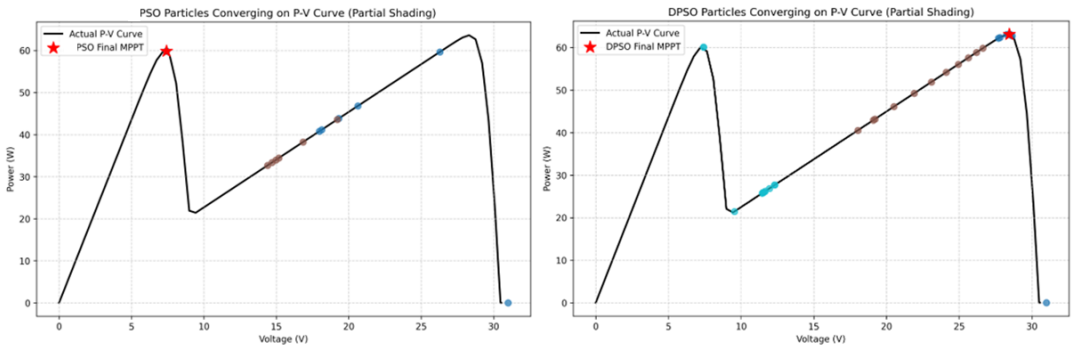


Figure 3.6: Graphs demonstration PSO convergence (left) and DPSO convergence (right)

was generated to find a benchmark P_{max}, and compared to the output from the hybrid tracker. The results can be seen in Figure 3.7, showing that the tracker matches the implicit solver.

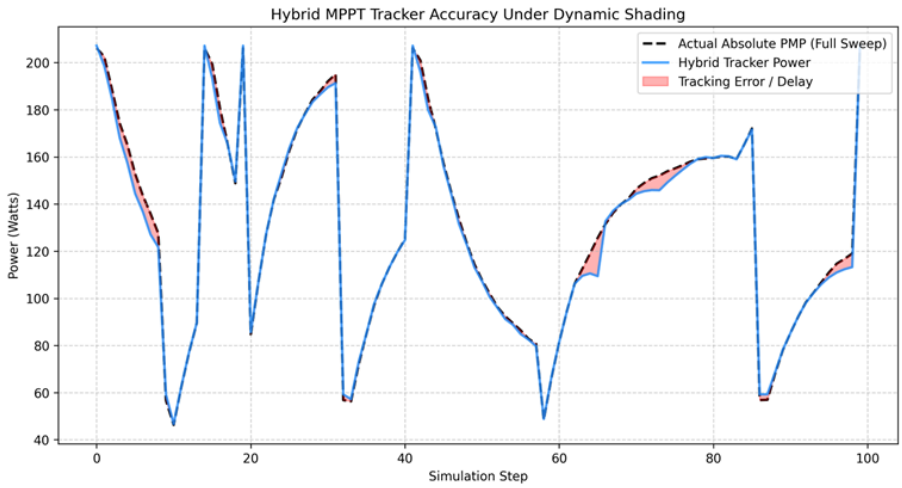


Figure 3.7: A graph showing the tracker compared to a full P-V graph for a power sweep

3.5 Pipeline 3 - Graphical Irradiance Gathering

With the backend solvers developed, the goal of this final pipeline is to implement the graphical frontend, which will work to gather irradiance data, package it and send it to the backend.

3.5.1 Reverse Ray-Casting

As discussed in Section 1.4, the ideal methodology for this section is distributed reverse ray-casting, which involves sending a series of rays from an object to detect shading. Unity was chosen to develop the frontend, as it had a global simulation manager, to manage the updates, and built in ray-casting and lighting methods.

First, a C# file `SolarCellTracker` was created, to manage the panel and ray-casting. This script would generate a panel as a grid of cells. At each step, it would randomly sample points on the surface of each cell, and fire a ray from these points towards the light source. From the percentage of rays from a cell that do not intersect with another object before the light, the irradiance for that cell can be individually interpolated between 0 and $1000W/m^2$. The angle of irradiance Equation 1.19, as discussed previously was implemented here, which calculated

the normals of the cells, the directions to the light, and found the cosine of the angle between them. This result was used to scale the irradiance in contact with each cell.

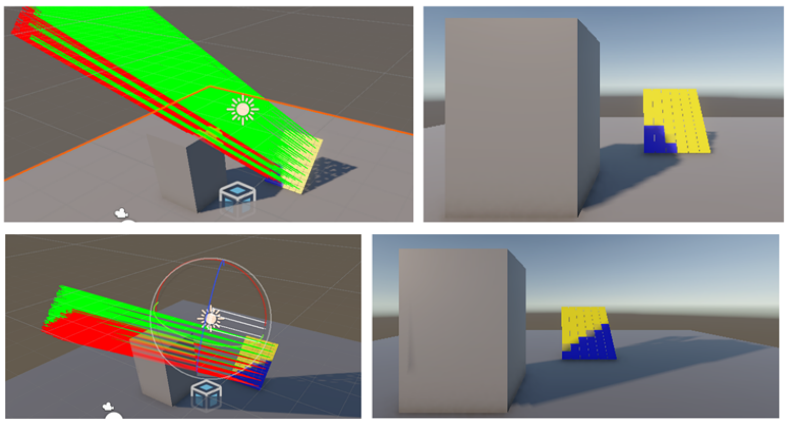


Figure 3.8: A figure showing ray casting results

In the demonstration (Figure 3.8) the rays (red for intersection and green for no intersection) and the colour of the cell (between blue and yellow based on irradiance) are shown. As shown towards the edge of the shadow, the cells are lighter than full shade demonstrating that they are receiving only partial shading. To simulate a light source moving like the sun, a C# script `LightController` was developed, to rotate the light source as the sun would.

3.5.2 The UDP Server

To connect the frontend to the python backend, a UDP client server model was selected, as it is standardised way to communicate between C# and python due to its speed and no need for a constant connection. The frontend would be the client, with the `SolarCellTracker` updated to calculate the shade of each cell and write the array to a local host socket every 0.1 seconds. The server socket would be set up in the python backend. Once initiated, it could read the irradiance arrays from the socket.

3.5.3 The Combined Process

The combined process was divided into two separately run sections. First the socket would be set up, and the frontend run to rotate the light source, calculating the irradiance array at each step, and writing to the socket. In this phase, the socket simply reads the array and writes it to a csv file. The second phase initiates the `Module` class and the `DPSO_MPPT` tracker, before reading the CSV file, and iterating through it. For each row, the cells in `Module` will be updated, and either the tracker or the explicit model used to calculate the P_{max} point. This was done to prevent the backend calculations slowing down the frontend.

3.6 Pipeline 3 - Validation

Pipeline 3 is validated in the Results 4.1.3, due to it being the final output of the project, and integrating the results of the previous two pipelines.

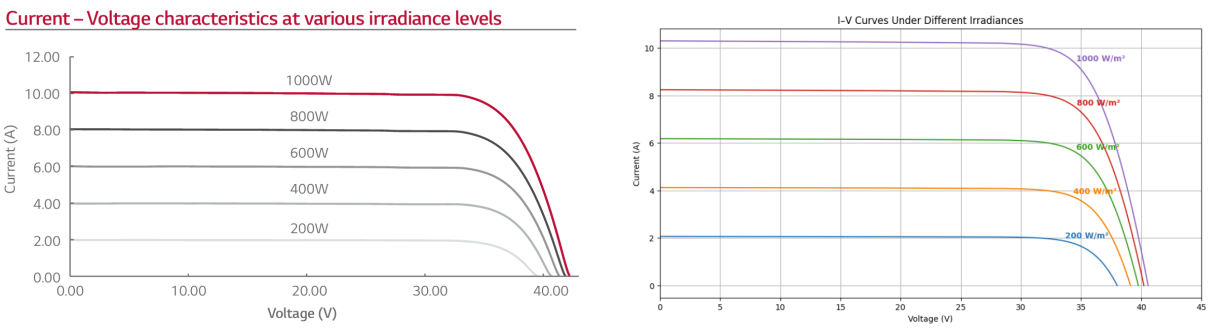
Chapter 4

Results, Evaluation and Discussion

4.1 Results

4.1.1 Panel Parameter Extraction

While in the validation section, the heuristic method (the chosen optimal method) was shown to have a low RMSE when predicting panels that exist inside the `PVLibs` database, to address the limitation discussed in Section 1.5, the method needed to be validated against a panel not in the database. For this, the panel LG ElectronicsInc LG330N1T V5 [35], was chosen, as it had I-V curves in the datasheet to validate against. The datasheet curves and the I-V curves built using the extracted parameters can be seen in Figure 4.1, with there being a slight discrepancy at I_{sc}. This is likely due to heuristic method increasing I_{sc} to induce convergence. However, this is only a marginal difference, and doesn't affect the P_{max} point, so can be ignored.

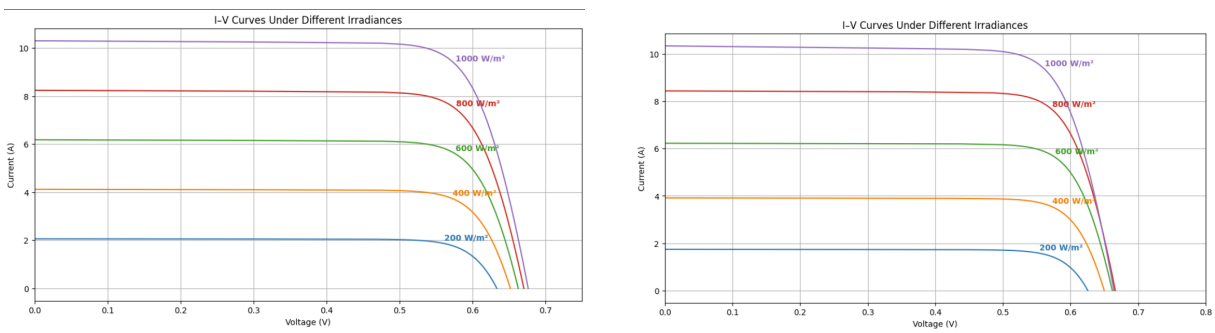


(a) Actual curves

(b) Predicted curves

Figure 4.1: I-V curves for a panel the panel: actual data (left) and predicted model (right).

4.1.2 Single Cell & ANN Prediction Results



(a) Full process results

(b) ANN model results

Figure 4.2: Comparative cell I-V curves.

Using the same panel, the I-V curves of the full single cell method, and the ANN prediction method were compared. As shown in Figure 4.2, there is a slight discrepancy at $100W/m^2$, with

the ANN predicting a lower I_{sc} than the heuristic method. This is likely due to there being less data at the lower end, as the dataset stops at $100W/m^2$, causing less accurate predictions.

4.1.3 Whole Module Results

This section focuses on comparing the performance and accuracy of the different models. Implicit was run simply generating different graphs, while explicit and the hybrid tracker were run using 1500 steps of irradiance data, with the results shown in Figure 4.3 (DPSO tracker) and Figure 4.4 (explicit model). Each step is equivalent to generating one P-V graph. As shown by Table 4.1, the implicit model was not particularly fast, and only reached a 75% accuracy. The explicit model and the tracker were 100x and 47x faster respectively.
However, there is a discrepancy between the two power-time graphs. The irradiance data starts at night, then the sun would rise for around 10 hours, before setting. Around hours 8-11 the panel will receive increasing shading, which rapidly disappears. The explicit model accurately finds its unshaded peak at hour 12.5, and the tracker doesn't. This is likely due to the tracker not detecting a large enough change in power and staying in `"local"` state instead of `"global"`.

Computational Model	**Theoretical Time**	**Average Execution Time**	**Accuracy vs.** `PVMismatch`	**Limitations**
Implicit P-V Solver (Trust Region)	$\mathcal{O}(V \cdot K(m \cdot n^2))$ (V = number of voltages tested)	~14 per graph	6/8 graphs	Doesn't accurately predict whole graph and relatively slow
Explicit P-V Solver	$\mathcal{O}(n)$ (n = number of cells to calculate)	~0.14 per step	8/8 graphs Data time graph accurate	The same accuracy issue as implicit at extreme shading
Hybrid Tracker (DPSO & P&O)	$\mathcal{O}(i \cdot K(m \cdot n^2))$ or $\mathcal{O}(1)$ (i = number of DPSO iterations)	Average ~0.3 per step	Data time graph mismatch at hours 11–16	Slightly slower than explicit, but may not always correctly find global mode

Table 4.1: Benchmarking of different methods

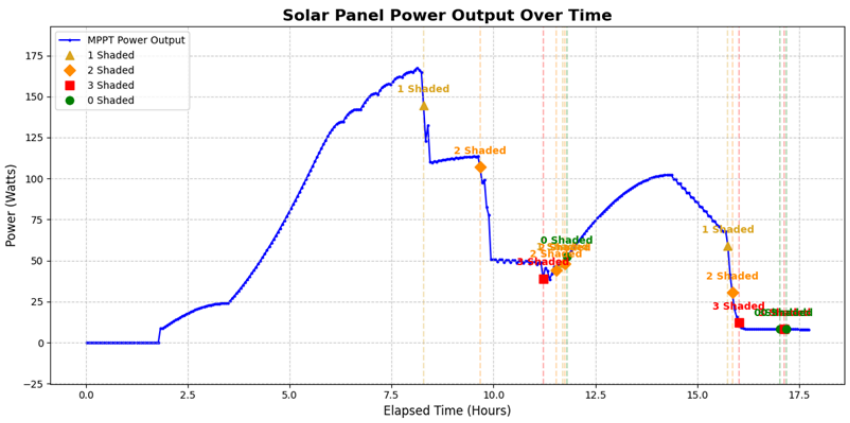


Figure 4.3: The power over time data, using the DPSO tracker to generate the graph

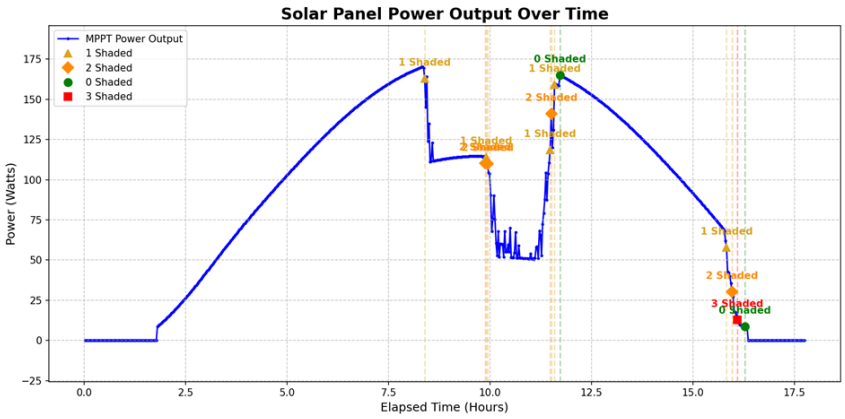


Figure 4.4: The power over time data, using the explicit model to generate the graph

4.1.4 Combined Process Results

In order to test snapshots of the combined data gathering and prediction method `results_gathering.py` was written to open the UDP socket, take in a single step of data from the frontend. It would then run the DPSO, implicit, explicit and `PVMismatch` solver to benchmark. The results are shown in Appendix D. These graphs generally show that the methods work, with the shape of the curves mostly matching `PVMismatch`. They further show the DPSO method consistently finds the peak of the implicit curve. Interestingly, while the explicit model better demonstrates the expected P-V curve shape, the implicit model tends to closer match the expected P_{max}.
The results also showed the irradiance angle equation working correctly, with a difference of $150W$ between a directly above sun and a sun at sunset position. Finally, the results expose a limitation of the models: no matter how heavy the shading is, the power output never goes to 0. This is likely due to the dataset used to train the ANN not having any results for $0W/m^2$ irradiance, meaning the ANN will always predict the parameters at their minimum $100W/m^2$.

4.2 Evaluation

Objectives Overall, the project completed the 5 main objectives set out in Section 1.1. It investigated different methods to predict cell parameters for the single diode model, developed computational solvers to generate P-V curves for a whole module using trust region and explicit methods, and implemented and tested different max power point tracking algorithms. In addition, a graphical method was used to calculate shade data, and this data used to build experiments, in order to test our models against pre-existing models to ensure accuracy.

Final Solution Design Based on the results and performance testing, the optimal structure for the backend is to use the heuristic method to build a dataset of cell level parameters, build an ANN with 2 hidden layers and 33 nodes and then use the explicit model to find the panels P_{max}. In terms of the frontend, the reverse ray casting method are demonstrated to work as expected. The UDP client server structure worked best with the 2 phase approach as discussed in Section 3.5.2, where the data gathering and power tracking are run as separate processes.

Accuracy and Efficiency The 2 main constraints for this project were efficiency and accuracy, meaning developing a model that ran as accurately as possible in minimal time. I believe this goal was met with the explicit model being able to model a whole day in 3 minutes, and with the ANN finding a compromise between accuracy and performance. However, as noted, the accuracy of the ANN prediction fails at the lower end, and the heuristic method that builds the database shows discrepancies at the I_{sc} point.

4.3 Future Work

Physical Cell Testing To improve the accuracy of the ANN, a larger dataset could be built, including more extreme values at the lower end. Furthermore, physical cell and panel testing could ensure correct prediction. Monitoring either cell or panel level parameters at different irradiances and temperatures could have three uses. First of all, it could allow better formed heuristics for the heuristic method. Secondly, if testing was precise, it could completely replace the heuristic method, by directly building a dataset that the ANN could use. Finally, it could be used to further validate the graphical experiments in Appendix D.

A Faster Model While the explicit model was fast, 3 minutes to run a simulated day may still not be fast enough. Having demonstrated how much the hybrid tracker improved the implicit model, it is a reasonable assumption that it could similarly increase the performance of the explicit model. By restructuring the tracker to test currents to find their equivalent power, the tracker could reduce the number of full current sweeps the explicit model has to perform, by replacing most sweeps with P&O steps.

Cell Temperature Modelling In addition, while the model can account for changing cell temperature, as demonstrated by Section 3.2.2, the temperature of the cells are never actually calculated, and assumed to be a constant 25°C. An improvement would be to model the temperature in the graphical model, based on time of day, and also in the backend by the resistance of the cell. The higher the resistance of the cell, the more heat that should be generated, especially around hot spots, where power is dissipated in reverse biased cells.

Improved Ray Tracing With more powerful hardware, a more advanced version of the ray-casting graphical model could be developed. In place of reverse ray-casting, ray-tracing could be used. Ray-tracing would track the way light reflects off surfaces, and the angle these reflections hit objects at. This could be used to calculate both diffuse and reflected irradiation, instead of just direct beam, further improving the accuracy of the irradiance data. Again the accuracy of the frontend model could benefit from experimenting with a physical panel, to validate the output.

A Continuous Pipeline Finally, with a more efficient backend, the two stage processing method implemented in Section 3.5.2 could be replaced with a single stage, where the backend reads from the socket and updates the power tracker in real time. This would allow the P_{max} to be seen simultaneously with live updates from the frontend.

References

[1] P Rappaport. The photovoltaic effect and its utilization. *Solar Energy*, 3(4):8–18, 1959.

[2] A Woyte, R Belmans, and N Johan. Partial shadowing of photovoltaic arrays with different system configurations: literature review and field test results. *Solar Energy*, 74(3):217–213, 2003.

[3] C. Honsberg and S. Bowden. Module circuit design, 2019. [Online] available at https://www.pveducation.org/pvcdrom/modules-and-arrays/module-circuit-design.

[4] C. Honsberg and S. Bowden. Shading, 2019. [Online] available at https://www.pveducation.org/pvcdrom/modules-and-arrays/shading.

[5] C. Honsberg and S. Bowden. Mismatch for cells connected in series, 2019. [Online].

[6] C. Honsberg and S. Bowden. Bypass diodes, 2019. [Online] available at https://www.pveducation.org/pvcdrom/modules-and-arrays/bypass-diodes.

[7] V Lo Brano and G Ciulla. An efficient analytical approach for obtaining a five parameters model of photovoltaic modules using only reference data. *Applied Energy*, 111:894–903, 2013.

[8] J Park, C Yu, S Min, D Shim, and J Shin. Extracting the inherent ideality factor of a diode from electrical current–voltage characteristics. *Electronics Letters*, 59(23), 2023.

[9] J. J. More and D. C. Sorensen. Newtonx27;s method. Technical report, Argonne National Lab., IL (USA), 02 1982.

[10] N. Blair, A Dobos, J Freeman, Ty Neises, M. Wagner, T. Ferguson, P. Gilman, and S. Janzou. System advisor model, sam 2014.1.14: General description. Technical report, National Renewable Energy Laboratory, 2013.

[11] T Ahmad, S Sobhan, and Md. F Nayan. Comparative analysis between single diode and double diode model of pv cell: Concentrate different parameters effect on its efficiency. *Journal of Power and Energy Engineering*, 4(3):31–46, 2016.

[12] Jinko Solar. Jkm550-570m-72hl4 module datasheet. Technical report, Jinko Solar, 2023.

[13] A El Tayyan. A simple method to extract the parameters of the single-diode model of a pv system. *Turkish Journal of Physics*, 37(1):121–131, 2013.

[14] D Protić and M Stanković. The q-levenberg-marquardt method for unconstrained nonlinear optimization. *arXiv preprint arXiv:2107.03304*, 2021.

[15] W De Soto, S Klein, and W Beckman. Improvement and validation of a model for photovoltaic array performance. *Solar Energy*, 80(1):78–88, 2006.

[16] A Dobos. An improved coefficient calculator for the california energy commission 6 parameter photovoltaic module model. *Journal of Solar Energy Engineering, Transactions of the ASME*, 134(2), 2012.

[17] E Karatepe, M Boztepe, and M Çolak. Development of a suitable model for characterizing photovoltaic arrays with shaded solar cells. *Solar Energy*, 81(8):977–992, 2007.

[18] J. Zou, Y. Han, and SS. So. Overview of artificial neural networks. *Methods in Molecular Biology™*, pages 14–22, 2008.

[19] A Mohapatra, B Nayak, P Das, and KB Mohanty. A review on mppt techniques of pv system under partial shading condition. *Renewable and Sustainable Energy Reviews*, 80:854–867, 2017.

[20] W Zhang, R Zhang, R Shang, J Li, and L Jiao. Application of natural computation inspired method in community detection. *Physica A: Statistical Mechanics and its Applications*, 515:130–150, 2019.

[21] Robert W. Erickson and Dragan Maksimovic. *Fundamentals of Power Electronics*. Kluwer Academic Publishers, Norwell, MA, 2nd edition, 2001.

[22] K Lian, J Jhang, and I Tian. A maximum power point tracking method based on perturb-and-observe combined with particle swarm optimization. *IEEE Journal of Photovoltaics*, 4(2), 2014.

[23] J Bishop. Computer simulation of the effects of electrical mismatches in photovoltaic cell interconnection circuits. *Solar Cells*, 25(1):73–89, 1988.

[24] P Eggers. Parallelization of ray casting for solar irradiance calculations in urban environments. Bachelor's thesis, University of Gävle, Faculty of Engineering and Sustainable Development, Gävle, Sweden, 2021. Available at DiVA portal https://hig.diva-portal.org/smash/record.jsf?pid=diva2

[25] A. Kalogirou. *Solar Energy Engineering: Processes and Systems*. Academic Press, Oxford, UK, 2 edition, 2013.

[26] R Cook, T Porter, and L Carpenter. Distributed ray tracing. *Computer Graphics*, 18(3):137–145, 1984.

[27] W Holmgren, C Hansen, and M Mikofski. Pvlib python: a python package for modeling solar energy systems. *Journal of Open Source Software*, 3(29):884, 2018.

[28] M Mikofski, B Meyers, G Kimball, et al. PVMismatch: a free open-source python package for simulating full pv system current-voltage curves. `https://github.com/SunPower/PVMismatch`, 2018.

[29] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, et al. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020.

[30] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011.

[31] H Krekel et al. pytest: helps you write better programs, 2004. Accessed: 2026-04-27.

[32] Canadian, Solar Inc. Preliminary technical information sheet high density mono perc module. Technical report, Canadian Solar Inc., 2023.

[33] T Akiba, S Sano, T Yanase, T Ohta, and M Koyama. Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019.

[34] C Harris, K Millman, and S van der Walt. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.

[35] LG Electronics. Lg neon 2 bifacial (60 cell) 330w-340w v5 datasheet. Technical specification, LG Solar, 2019.

Appendix A

Self-appraisal

A.1 Critical self-evaluation

Looking at the results of the project against the aim, I feel I have achieved the goal of developing a model capable of simulating photovoltaic modules and their output, based on shading mismatch. The model can not only predict where shade on a panel will fall, but also estimate the prospective power output based on such shading. I believe that strong personal adaptability has been key to my completion of this project. There were several points where a step in the project failed, and a new approach had to be tested. The original parameter curve fitting, the voltage first model and the live frontend update/backend calculation model all had issues that needed new approaches.

However, an improvement for this project would have been a more comprehensive literature review, potentially covering more alternative routes to development. For example, studying how `PVLibs` handled the parameter estimation or how `PVMismatch` accounted for bypass diode activation. Finding more alternatives like this would've allowed for a smoother development, as they would have already been written up in the literature review. As it was, alternatives had to be investigated on the fly during development.

Furthermore, a big challenge was the efficiency of the solution. The original goal of the project was to run the power tracking live alongside the graphical simulation, outputting the panel power in real time. With the efficiency problems, this wasn't possible with local hardware. Given more time, deploying the model to a distributed system to run in place of a laptop and rewriting the code in a compiled language, such as `C++`, could reduce interpretation and calculation latency. This would ideally increase the performance and potentially meet the goal of real time simulation.

A.2 Personal reflection and lessons learned

The most important lesson I learned from this project was the need for contingency planning and iterative processes. Initially, the aim was to have completed the research and literature review, prior to beginning the implementation. However, as mentioned there were frequent issues, which required a reset and a new method. This would mean I would have to research different methods and attempt to refactor based on them, which slowed development progress. The process would have been much more fluid, if there had been backup plans and alternate methods in place before development even started.

I believe that my project development wasn't as structured and effective as it could have been. While branches, version controls and Git issues were used to organise deliverables, more frequent commits could have aided the clarity of development. As it was, commits were only made after large sections of the project had been completed, potentially making the development harder to track. As an example, one commit was for the completion of the final

MPPT technique. Adopting atomic commits, where the development of each method's algorithm, unit testing and hyperparameter optimisation would be separated, would have improved version control and development traceability.
While unit testing was used for different stages, and validation against research papers and various models was used, I think that an earlier adoption of a continuous integration (CI) pipeline paired with more frequent commits would be an improvement. Earlier in the project, before the pipeline was setup, these unit tests had to be run manually prior to commits. While basic unit tests validated the parameter outputs against PVLib, there was limited testing to ensure that all of the functions ran without error. The project would have benefited from Test Driven Development (TDD) using automated regression tests. A series of regression tests making sure functions such as the MPPT or I-V curve generation still ran without errors, which could be automatically run prior to a merge, using the CI pipeline, would have reduced the number of issues during development. Furthermore, some of the tests run in the CI pipeline were using the implicit model, which is shown to not always be accurate, so the automated tests would fail. Instead of including validation, the tests should have simply focused on lack of run time errors.
From a technical point of view, I have gained an insight into how complex mathematical problems are solved for simulations. I have learned how to use trust region methods to solve simultaneous equations with large amounts of unknown values; artificial neural networks to predict parameters based on data, and particle swarm optimisation methods to find graph peaks. In addition, I have learned about the electrical engineering of solar panels, including panel structure, circuit layouts and operation.

A.3 Legal, social, ethical and professional issues

A.3.1 Legal issues

This project interacts with data and information about photovoltaic modules, for example the datasheets that contain module specifications. Some companies may choose to keep this information proprietary. To ensure that copyright is adhered to, the panels and datasheets chosen need to be publicly available. The same standard applies to the libraries used for development and validation. These libraries (e.g. PVMismatch and PVLibs) require acknowledgement of their usage in a project, adhering to their open source licenses.

A.3.2 Social issues

Modern social issues exist around rising energy costs and desire for more renewable energy, therefore a simulation that could accurately predict power output of a panel given potential shading conditions would be useful to inform layout and viability of solar panels. A user could use such a model to help decide whether they would get a large enough return from investing in solar panels. While this should have positive social impacts, it makes the accuracy even more important. If the model tells a user they will receive a higher output than they will in actuality, this may negatively impact a person's life, as they would have spent a large sum of money, without the expected gains in power and reduction in electricity bills.

A.3.3 Ethical issues

With recent concerns about the impact of more traditional fossil fuels on the environment, renewable energy has become increasingly sought after. A common source of renewable energy are solar panels. There has been a recent trend of homeowners incorporating solar panels into their homes, in order to save money on increasing electricity bills, however this is often expensive (can be between £5-6000 for an average house). Therefore it is important to ensure that the panels will produce enough power to compensate for this cost. For example, a roof facing east or west or surrounded by tall trees, will receive less sunlight, and not end up being worth the money.

Although this project aims to be able to demonstrate the impact of shade on a solar panel, it should not be used to inform real world decisions. This is an academic research project, rather than an industry tool, meaning it had limited time for testing and performance improvements. In reality, it would require much more strenuous testing and validation against real world data, before deployment and usage. There is an additional risk of this project not taking into account the damage that can be done by incorrect placement of a solar panel. If a panel regularly receives shade on a single cell, it can generate a hotspot, due to the higher resistance. This increased temperature can damage the solar panel over time.

As this is an academic project, it is important to be open and honest about the algorithms and data. The project should not artificially tweak data to fit hypotheses better, or hide limitations in the simulation. In addition, the data used to train the ANN should be generated as described, to avoid biasing it towards better performing conditions, producing higher power outputs. Furthermore, some of the work done is based on others' research, therefore it is important to properly cite and credit where my work is inspired from. It is especially important to ensure that datasheets used for solar module parameters are open source, and the same with the data pulled from the PVLibs CEC database.

Finally, executing computationally heavy solvers can carry a measurable CPU energy cost themselves. However the project has focused on optimisation to limit this impact, and the project also helps to facilitate optimal solar panel deployment, offsetting the cost of running.

A.3.4 Professional issues

During the development of this project, I attempted to adhere to professional standards, namely the British Computer Society (BCS) code of conduct. Specifically, to maintain integrity, it was vital to ensure that the limitations of the developed models are documented, and not glossed over using artificial data. Due to the potential professional applications of the simulation, it was additionally important that the decisions I made throughout are understandable and transparent. The reasoning behind the chosen methodology at each stage needs to be clearly explained, evidenced and the model should be able to be reproduced from the report. To this end, each step has an in depth explanation, supported by produced data and figures when necessary.

Additionally, the simulation will require validation before commercial use, to ensure that it isn't returning false values. The external libraries used for this validation need to be cited and credited, and ideally testing against real solar panels would be performed prior to deployment.

Finally, for commercial use this project would require efficiency and further usability improvements. Deploying to a distributed system instead of local hardware would solve the majority of the efficiency issues. Before that, I would recommend further work to make it easier to set up the Unity graphical model and run the backend. A simple UI to place objects and panels, that could translate to Unity would be ideal, with a containerised and deployed server to handle the power calculations.

Appendix B

External Material

Material	Purpose	Link
Unity 3D	Development of the 3D graphical model to accurately simulate environmental shading and sunlight.	`https://unity.com/`
SciPy	Python library utilized for the `least_squares` and `fsolve` trust-region solvers.	`https://scipy.org/`
PVMismatch	Used to validate mismatched P-V curves.	`https://github.com/SunPower/PVMismatch/`
PVLibs	Used to validate panel 5 parameter extraction methods and for its CEC database.	`https://github.com/pvlib/pvlib-python`
PyTest	Creating the unit tests used in the CI pipeline.	`https://docs.pytest.org/en/stable/`
Pandas	Reading and writing irradiance and power data to CSV files.	`https://pandas.pydata.org/`
NumPy	For parallelising the explicit cell equations.	`https://numpy.org/`
Sci-Kit Learn	For developing and using the ANN for cell prediction.	`https://scikit-learn.org/stable/index.html`
Optuna	For running trials to find the optimal ANN structure	`https://optuna.org/`

Table B.1: Summary of External Materials and Software Resources

Appendix C

Supplementary Figures

C.1 Tables

C.1.1 Values and Bounds for Heuristic Method

The first 2 tables are results from [16] used to inform an initial guess for the heuristic based 5 parameter solver, and the last shows reasonable bounds for these parameters.

Technology	**Estimate for a**
Mono-crystalline Silicon (Mono-Si)	$0.027 \cdot N_\mathrm{s} - 0.0172$
Multi-crystalline Silicon (Multi-Si)	$0.026 \cdot N_\mathrm{s} + 0.0212$
Amorphous Silicon	$0.029 \cdot N_\mathrm{s} + 0.5264$
Cadmium Telluride (CdTe)	$0.012 \cdot N_\mathrm{s} + 1.356$
Copper Indium Gallium Selenide (CIGS)	$0.018 \cdot N_\mathrm{s} + 0.327$

Table C.1: Linear regression results for the modified ideality factor (a) estimate [16].

Technology	$C_\mathbf{s}$	$C_\mathbf{sh}$
Mono-crystalline Silicon (Mono-Si)	0.32	4.92
Multi-crystalline Silicon (Multi-Si)	0.34	5.36
Amorphous Silicon	0.59	0.92
Cadmium Telluride (CdTe)	0.46	1.11
Copper Indium Gallium Selenide (CIGS)	0.55	1.07

Table C.2: Average coefficients for the estimation of R_s and R_sh [16].

Variable	**Minimum**	**Maximum**
a	0.05	15.0
I_ph (A)	0.5	15.0
I_s (A)	1×10^{-16}	1×10^{-7}
R_s (Ω)	0.001	75.0
R_sh (Ω)	1.0	10000.0
Adjust (%)	-100.0	100.0

Table C.3: Parameters boundaries used as sanity checks during optimization [16].

C.1.2 Tracker Hyper Parameter Optimisation

These tables show the results from hyper parameter optimisation for PSO and DPSO methods.

Number of Particles	**Time (s)**	**Difference (W)**
2	5.95	2.65
3	10.03	1.07
4	15.37	0.84
5	14.28	0.84
6	17.62	0.58
7	19.63	0.21
8	23.23	0.40
9	32.26	0.11
10	34.80	0.34

Table C.5: Optimization of PSO particle count.

W	**C1**	**C2**	**Time (s)**	**Difference (W)**
0.4	0.3	0.3	16.23	0.38
0.4	0.3	0.4	14.05	0.52
0.4	0.3	0.5	18.08	0.14
0.4	0.4	0.3	15.81	0.45
0.4	0.4	0.4	14.20	0.52
0.4	0.4	0.5	17.96	0.14
0.4	0.5	0.3	14.85	0.47
0.4	0.5	0.4	14.32	0.52
0.4	0.5	0.5	17.91	0.14
0.5	0.3	0.3	18.82	0.14
0.5	0.3	0.4	17.91	0.20
0.5	0.3	0.5	12.06	1.41
...	...	...	...	...

Table C.4: Optimization of PSO weights.

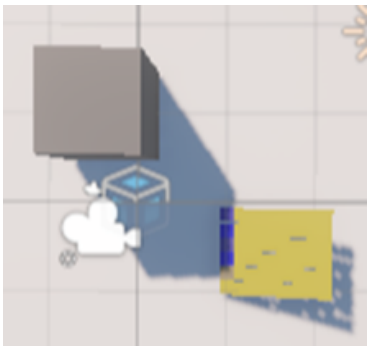
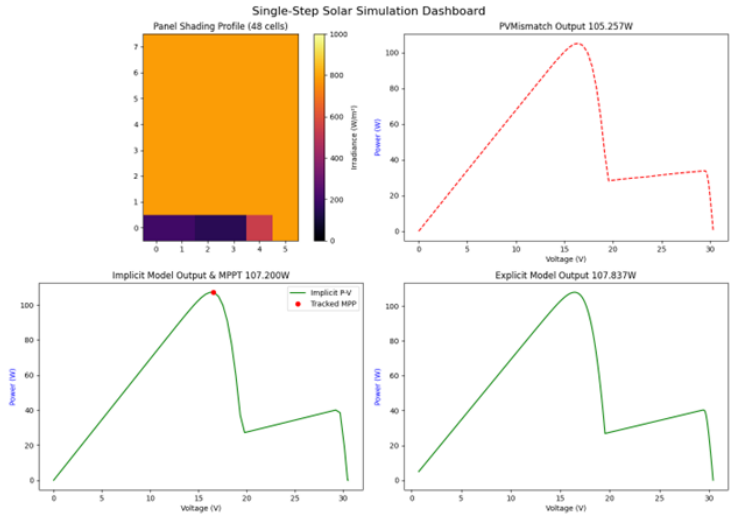
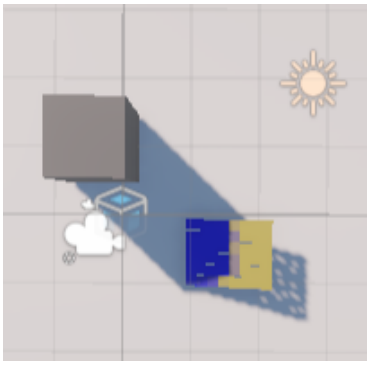
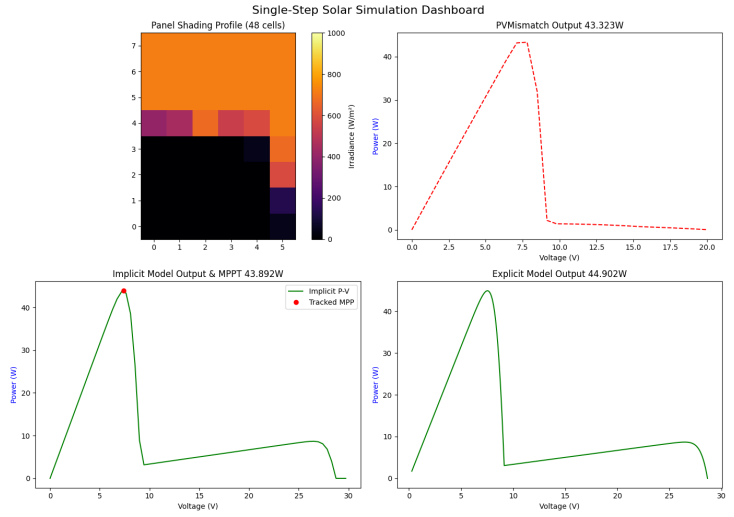
Particles	**Inertia (w)**	**Max Vel**	**Error (W)**	**Time (s)**	**Score**
3	0.3	0.25	0.0670	20.6	21.3
4	0.3	0.05	0.0708	23.3	24.0
5	0.3	0.05	0.2120	25.3	27.5
4	0.3	0.10	0.0707	27.1	27.8
5	0.7	0.05	0.2120	26.9	29.0
4	0.7	0.05	0.5260	23.8	29.1
4	0.5	0.05	0.0680	29.6	30.3
5	0.5	0.10	0.2120	28.4	30.5
3	0.3	0.15	0.9010	22.1	31.1
5	0.7	0.10	0.2120	29.4	31.5
3	0.5	0.15	0.3050	28.7	31.7
3	0.7	0.25	0.3020	28.8	31.8
5	0.3	0.10	0.1600	30.3	31.9
5	0.5	0.05	0.0723	35.2	35.9
3	0.7	0.15	1.5200	23.0	38.3
4	0.7	0.10	0.2160	37.0	39.1
4	0.5	0.10	0.0731	38.6	39.3
4	0.3	0.15	0.7760	36.3	44.1
5	0.3	0.15	0.7790	37.1	44.9
5	0.5	0.25	0.7770	37.3	45.1
5	0.3	0.25	0.7790	38.4	46.2
5	0.5	0.15	0.7850	39.1	46.9
4	0.3	0.25	0.7760	39.7	47.5
4	0.5	0.15	0.0724	47.4	48.2
4	0.5	0.25	0.0725	48.8	49.5
5	0.7	0.25	0.7700	45.6	53.3
4	0.7	0.15	0.9080	45.6	54.7
4	0.7	0.25	0.0412	56.5	56.9
5	0.7	0.15	0.7980	53.2	61.2
3	0.3	0.10	5.8000	14.5	72.4

Table C.6: A table showing DPSO hyperparameter optimisation over 3 different graphs.

Appendix D

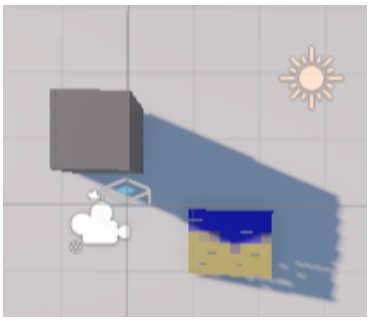
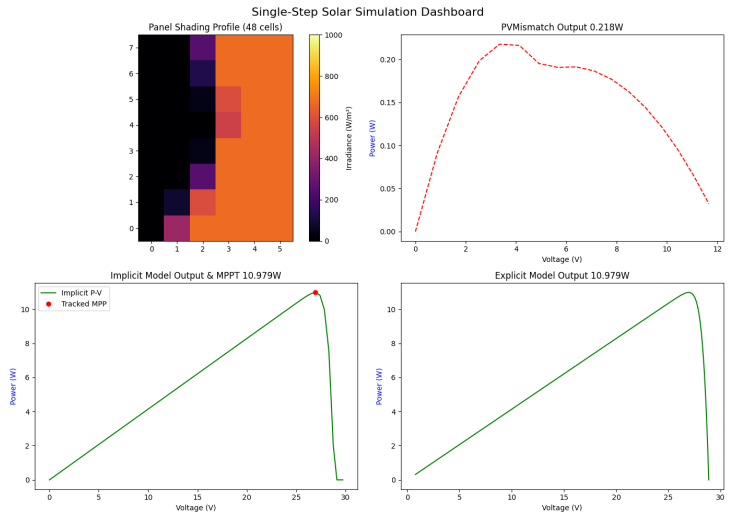
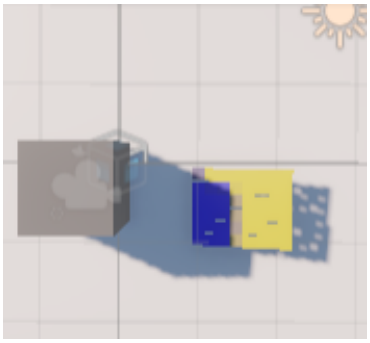
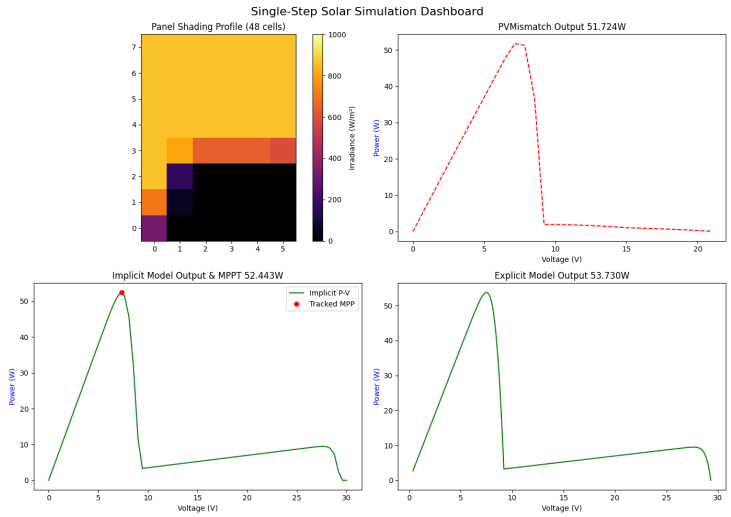
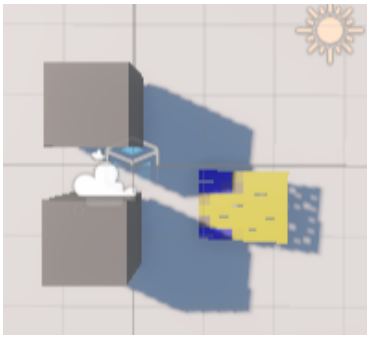
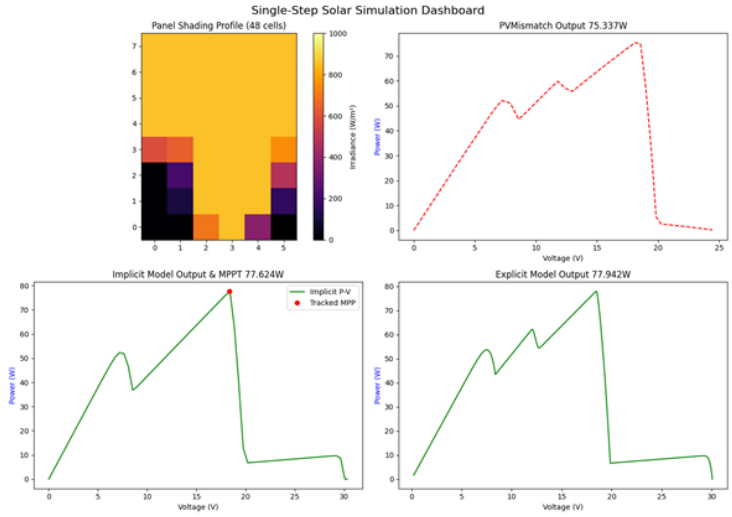
Supplementary Graphical Results

These are additional graphs to show the graphical method working. The left shows the shading generated by the graphical model and ray-casting. The right shows the irradiance array interpreted on a panel top left, the PVMismatch result in the top right, the implicit model and tracker in the bottom left, and the explicit model in the bottom right. The graphs were generated using Prism Solar Technologies Bi48 267BSTC.

Above View	**Results**
	
	

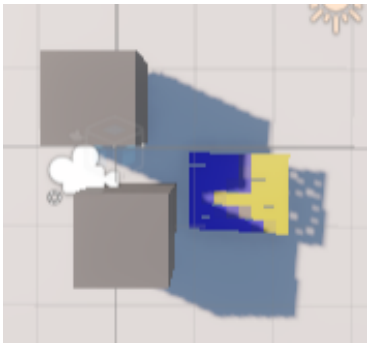
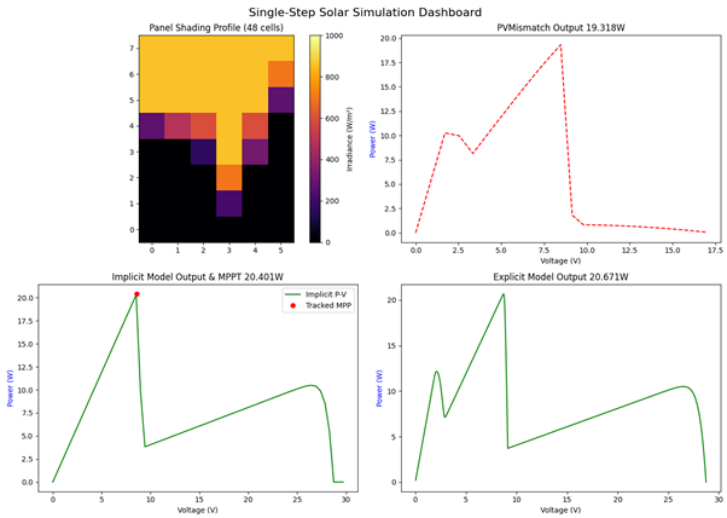
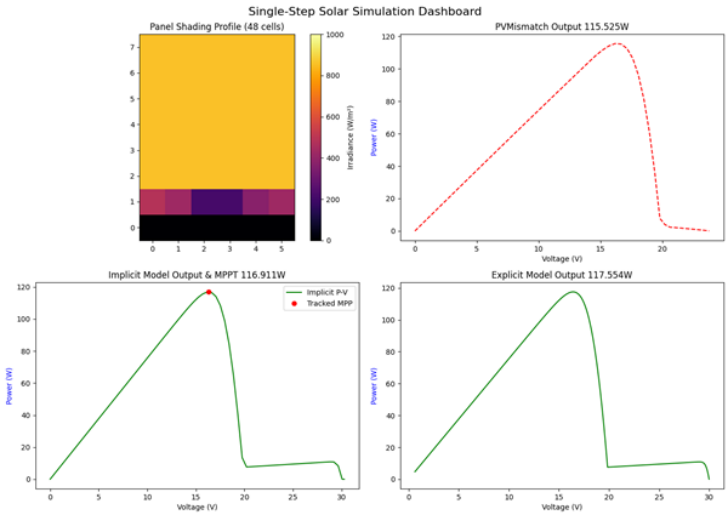
Continued on next page

Table D.1 – continued from previous page

View	Results
	
Difference seen here between PVMismatch and our models, when all 3 substrings experience partial shading. Large amount of mismatch would indicate a lower output as seen in PVMismatch.	
	
	

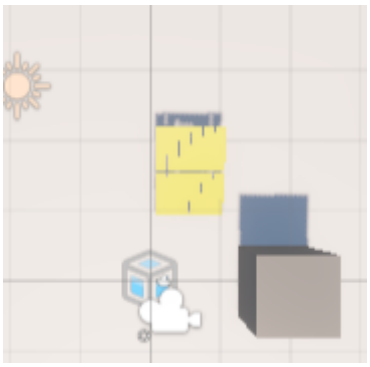
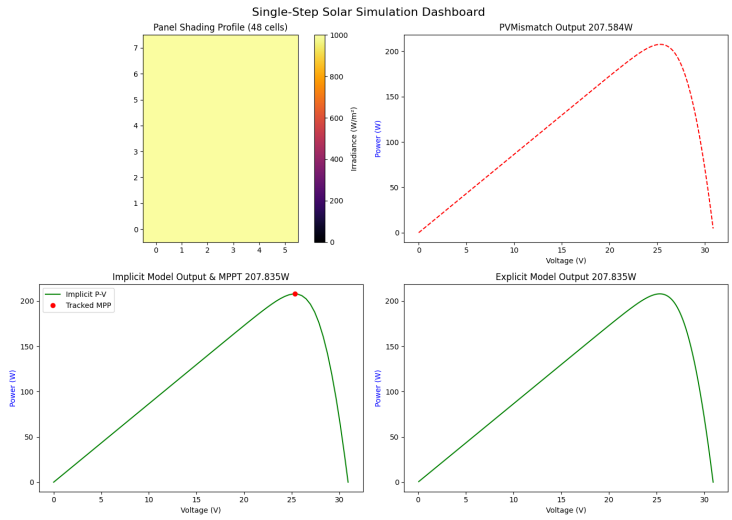
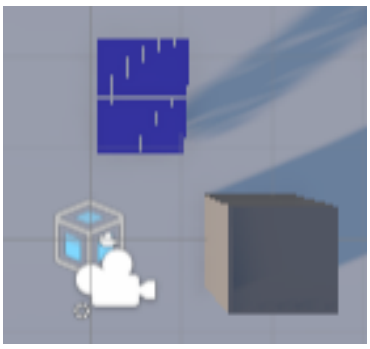
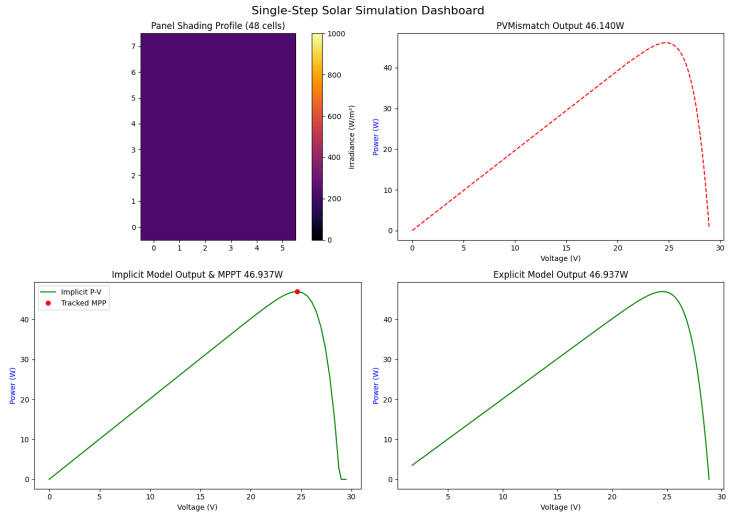
Continued on next page

Table D.1 – continued from previous page

View	Results
	
	
As shown by the last 3, the amount of mismatch between substrings informs the number of peaks. In the 1st there is the middle substring receiving minimal shading while the other 2 receive large amounts and 3 peaks, the 2nd has much closer shading on the 2nd substring and has 2 peaks, and the 3rd has identical shading and only one peak. This is picked up well by the explicit model but badly by the implicit model.	

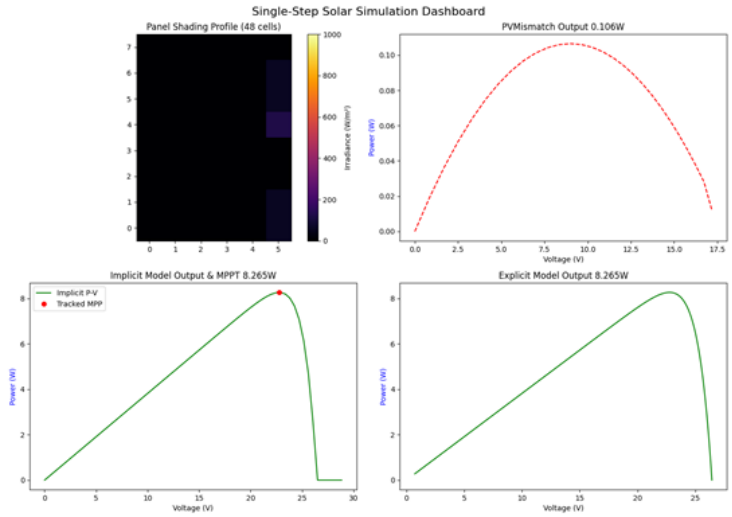
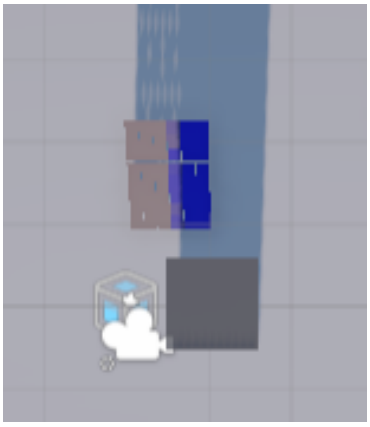
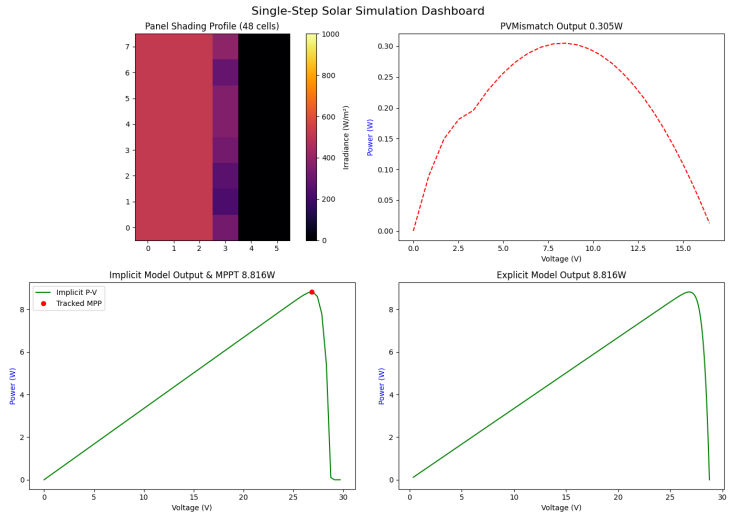
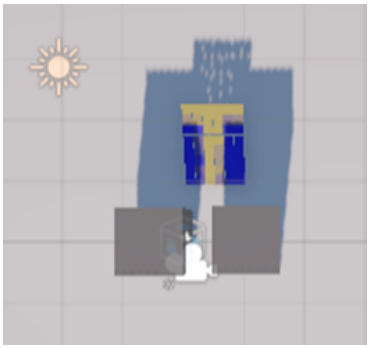
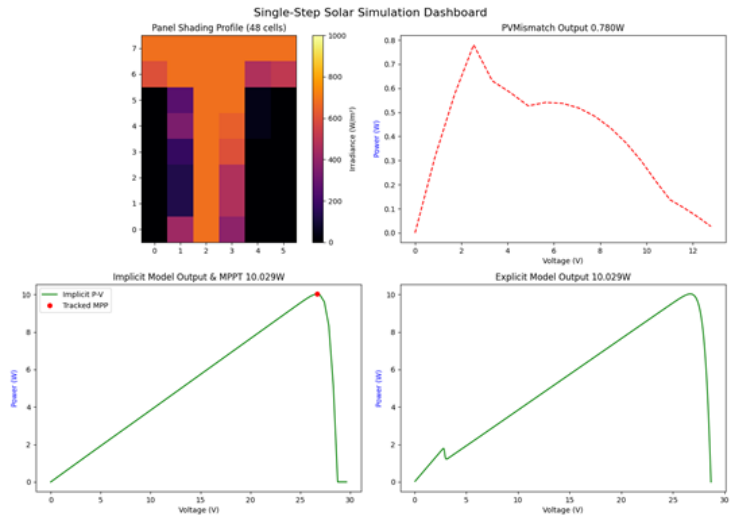
Continued on next page

Table D.1 – continued from previous page

View	Results
	
	
These 2 show the difference the angle of light makes. The 1st shows an angle closer to midday, with the light almost directly above as shown by the short shadows, while the second is closer to evening, with much longer shadows. The graphs and images demonstrate no shading, but an accurate difference of around 150W purely based on the time of day.	

Continued on next page

Table D.1 – continued from previous page

View	Results
	
	
	

Continued on next page

Table D.1 – continued from previous page

View	Results
These final 3 results show where our model tends to break down. During complete shading of an entire substring, as shown by the right substring being shaded in the 2nd, a minimal amount of power is produced in the PVMismatch model. This is due to any current passing through the panel causing potential damage with such a large mismatch. As demonstrated by the 1st result, where the panel is completely shaded, our modules still calculate some power being produced, when it shouldn't. Future work should involve some adjustments to the power calculations at the lower end.	